

First-Class Refinement Types for Scala

MATT BOVEL[✉], EPFL, Switzerland

VIKTOR KUNČAK, EPFL, Switzerland

MARTIN ODERSKY, EPFL, Switzerland

Refinement types—types qualified with logical predicates—have proven effective for lightweight verification in languages like Liquid Haskell, F*, and Dafny. However, in these systems refinements are either written in a separate specification language or treated as second-class annotations, disconnected from the host language’s type system. This disconnect creates usability barriers: programmers must maintain two mental models, and refinements cannot interact with features like type inference, subtyping, or overloading.

We present the design of first-class refinement types for Scala 3, where refinements are ordinary types that participate in subtyping, inference, and pattern matching alongside existing language features. We prove type soundness of a core, pure calculus mechanized in Rocq, combining dependent function types, bounded polymorphism, positive equi-recursive types, union and intersection types, and refinement types, using a fuel-bounded definitional interpreter and semantic typing. A distinctive design choice is our partial-correctness semantics: predicates are arbitrary terms that may diverge, and type soundness requires no termination assumptions. Finally, we implement our design as a prototype extension of the Scala 3 compiler with a lightweight e-graph-based solver for predicate entailment.

CCS Concepts: • **Software and its engineering** → **Compilers; Formal software verification; Data types and structures**; • **Theory of computation** → **Type theory; Program semantics**.

Additional Key Words and Phrases: Refinement Types, Scala, Definitional Interpreter, Semantic Types, Rocq

ACM Reference Format:

Matt Bovel, Viktor Kunčák, and Martin Odersky. 2026. First-Class Refinement Types for Scala. *Proc. ACM Program. Lang.* 10, OOPSLA2, Article 409 (October 2026), 29 pages. <https://doi.org/10.1145/3839541>

1 Introduction

A *refinement type* restricts a base type to values satisfying a logical predicate. For example, the type $\{x : \text{Int} \mid x > 0\}$ denotes the positive integers. Logical refinements as subsets of values have a long history [20], going back at least to Cartwright [6]. Freeman and Pfenning [12] coined the term *refinement types* for a more restricted mechanism distinguishing subtypes of algebraic datatypes by constructor structure. Flanagan [11] returned to general predicate refinements, checking them statically when possible and dynamically otherwise (*hybrid type checking*). Rondon et al. [33] then made predicate refinements more practical with *Liquid Types*, which automatically infer refinements via predicate abstraction and discharge proof obligations with SMT solvers. Systems built on these ideas such as Liquid Haskell [44], F* [39], and Dafny [27] have since verified properties ranging from safe array indexing [33] to the functional correctness of production cryptographic libraries [50].

The appeal of refinement types is that refinements are just *types*: they are checked statically, composed through ordinary type constructors, and propagated by standard typing rules. For instance, retrieving an element from a list of non-empty strings yields a non-empty string by

Authors’ Contact Information: Matt Bovel, EPFL, Lausanne, Switzerland; Viktor Kunčák, EPFL, Lausanne, Switzerland; Martin Odersky, EPFL, Lausanne, Switzerland.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/10-ART409

<https://doi.org/10.1145/3839541>

parametric polymorphism alone. More fundamentally, refinement types can be seen as [19] a generalization of Floyd-Hoare logic in which invariants are attached to *types* rather than *program points*, delegating abstraction and instantiation to the type system using subtyping and type inference. In practice, programmers can specify and verify properties using the type system they already know, with the same error messages, IDE support, and mental model.

However, these benefits are often diminished because existing refinement type systems process refinements in a *separate phase*, disconnected from the host language’s type checker. In Liquid Haskell, for instance, refinements are written inside special comments and verified by an external tool. Note how the type of `x` is declared twice in the following snippet: once as `Int` for the Haskell type checker, and once as `{v:Int | v mod 2 == 0}` for Liquid Haskell:

```
{-@ x :: {v:Int | v mod 2 == 0} @-}
let x = 42 :: Int in ...
```

As a result, refinement types are not “just types”: programmers must maintain two mental models, one for the host type system and one for the refinement layer, with separate error messages, separate IDE support, and limited ability to debug failures across the boundary. A recent usability study by Gamboa et al. [14] confirms this: one participant observed that “comments are usually seen as just optional information in the code and not something that is directly used by the compiler,” and another remarked, “It’s sort of like you’re doing two things at once because you’re implementing in Haskell. But you’re also talking to GHC, but you’re also talking to LiquidHaskell.”

The consequences of this separation run deeper than usability. Because the refinement checker operates after or alongside the host type checker, it must duplicate much of the host’s type inference machinery, an engineering burden that also limits expressiveness. Schmid and Kunčák [35] implemented qualified types for an earlier version of the Scala compiler (Dotty) with a refinement checker “largely independent of Scala’s own type checker.” This architecture meant refinement types could not interact with Scala’s type inference (they could not be inferred as type arguments in generic code) and required a separate qualifier inference algorithm that proved difficult to scale.

Userland Scala libraries such as Iron [18] and Refined [41] take a different trade-off: they encode refinement-like checks using opaque types and implicit evidence, benefiting from Scala’s own type inference and tooling, but at the cost of expressiveness; refinement proofs are limited to what Scala’s implicit resolution can discharge, ruling out dedicated efficient decision procedures for equality, arithmetic, ordering, or other domain-specific theories.

1.1 First-class Refinement Types

We show that refinement types can be first-class types in a mainstream programming language. Rather than adding a separate refinement layer, we extend the Scala 3 type checker itself so that refinement types are ordinary Scala types, checked during the initial type-checking phase and flowing naturally through subtyping and type inference. The Liquid Haskell example above becomes:

```
val x: (Int with x % 2 == 0) = 42
```

Here, `Int with x % 2 == 0` is a regular Scala type: it participates in subtyping, generics, pattern matching, type bounds, overloading, and implicits, eliminating the two-mental-models problem. There is one type system, one set of error messages, and one inference algorithm.

Integrating refinement types into the host language’s type system is not entirely new: Dafny [27] and F* [39] already treat refinement types as first-class citizens of their type checkers, and use subtyping to relate a refinement type to its base type. However, these languages were designed from the ground up around verification. To our knowledge, ours is the first integration of refinement

types into a pre-existing language where subtyping is central and pervasive, relied upon by classes, traits, mixin composition, path-dependent types, bounded polymorphism, overloading, and implicit resolution, while preserving the type system's existing behavior.

As a concrete example, consider Scala's bounded type polymorphism. In the snippet below, the ordering is defined for `Int` only, yet `maximum` applied to even numbers should return `Even`; this challenge motivated *abstract refinements* [43]. In our system, bounded polymorphism and subtyping suffice: `List(2, 4, 6)` is typed as `List[2 | 4 | 6]`, type inference instantiates `T` to `Int`, whose `Ordering` instance performs the comparisons, and `U` to `2 | 4 | 6`, and the result checks against `Even` since each literal satisfies the predicate:

```
def maximum[T: Ordering, U <: T](xs: List[U]): U = xs.reduce(max)
type Even = {v: Int with v % 2 == 0}
def example1: Even = maximum(List(2, 4, 6))
```

Fig. 1. A refinement type interacting with bounded type polymorphism.

As another example, consider overload resolution, a feature characteristic of object-oriented languages. Below, we define two overloads of `min`: one for sorted lists and one for arbitrary lists. Because refinement types participate in overload resolution, the compiler dispatches `min(1)` to the more specific overload when `1` is known to be sorted:

```
def min(l: List[Int] with l.isSorted): Int = l.head // O(1)
def min(l: List[Int]): Int = l.min // O(n)
def example2(l: List[Int] with l.isSorted): Int = min(l) // calls the first overload
```

Fig. 2. Overload resolution with refinement types.

1.2 Contributions

These examples illustrate a broader point: when refinement types are first-class, they compose freely with existing language features rather than requiring bespoke extensions for each interaction. We demonstrate this idea end-to-end, through the following contributions:

- We present the **design** of first-class refinement types in Scala 3: their syntax, semantics, and interaction with Scala's existing type system, including subtyping, type inference, bounded polymorphism, overloading, and implicits (§2). A distinctive design choice is a partial-correctness semantics for predicates: predicates are ordinary, possibly diverging pure terms whose meaning is defined operationally rather than by translation into a classical logic. A refinement holds if its predicate evaluates to true whenever it terminates (§2.3).
- We prove **type soundness** of a core, pure calculus capturing the essential features of our design, mechanized in Rocq [40] (§3). The operational semantics is given by a fuel-bounded definitional interpreter, and soundness is established via semantic typing, where types are interpreted as predicates on values. To our knowledge, this is the first mechanized soundness proof that combines refinement types with union and intersection types, bounded polymorphism (with lower and upper bounds), and positive equi-recursive types, a combination needed to model Scala's type system but not covered by prior mechanizations [3, 16, 38].
- We describe a **prototype implementation** in the Scala 3 compiler (Dotty), showing that refinement types can be added to an industrial-strength compiler with modest changes to its architecture (§4). We also present a lightweight e-graph-based solver for predicate entailment and evaluate our system on a suite of benchmarks (§4.4).

2 Design

In this section, we present the syntax (§2.1), define the predicate language and its purity restrictions (§2.2), discuss the partial-correctness semantics (§2.3), explain how expressions are given refinement types through equality facts and selfification (§2.4), and describe run-time checks (§2.5).

2.1 Syntax

We illustrate the syntax with the classic length-indexed vectors (Figure 3).

```

type Vec[T]
object Vec:
  def fill[T](n: Int, v: T): {r: Vec[T] with r.len == n} = ???
extension [T](a: Vec[T])
  def len: Int = ???
  def concat(b: Vec[T]): {r: Vec[T] with r.len == a.len + b.len} = ???
  def zip[S](b: Vec[S] with b.len == a.len): {r: Vec[(T, S)] with r.len == a.len} = ???
def example3(n: Int, m: Int): {r: Vec[(String, Int)] with r.len == m + n} =
  val v1 = Vec.fill(n, 0)
  val v2 = Vec.fill(m, 1)
  val v3 = v1.concat(v2)
  val mPlusN = m + n // inferred as Int
  Vec.fill(mPlusN, "").zip(v3)

```

Fig. 3. Length-indexed vectors using refinement types.

A refinement type has two syntactic forms: a *long form* with an explicit binder, and a *short form* that reuses a name from context.

Long form. The long form $\{x: T \text{ with } p(x)\}$ introduces an explicit binder x for the refined value. This is necessary when no name is already available, such as the return type of `fill` in Figure 3. The curly braces and explicit binder mirror set-builder notation and are visually consistent with other brace-delimited constructs in Scala.

Short form. When the refined value already has a name, as in a `val` or parameter declaration (as for the `b` parameter of `zip` in Figure 3), the binder can be omitted. The short form $T \text{ with } p$ reuses the enclosing binding’s name and desugars to the long form:

```

val x: (Int with x % 2 == 0) = 42
// desugars to:
val x: {v: Int with v % 2 == 0} = 42

```

Grammar. Refinement types extend Scala’s type grammar [7, §13] with two productions:

Type	::= ...	
	InfixType `with' PostfixExpr	-- short form
SimpleType	::= ...	
	`{ ' id `:' InfixType `with' Expr `}'	-- long form

Notably, predicates reuse Scala’s existing expression syntax: they are syntactically ordinary expressions, though subject to semantic restrictions (§2.2). The base type is an `InfixType`, which excludes complex type forms such as `match` types without parentheses.

2.2 Purity

Predicates are restricted to a pure fragment of Scala’s expression language: constants, stable identifiers (references to `val` bindings and parameters), field selections over `val` fields, term and type applications, constructors of case classes without initializers, boolean connectives (`&&`, `||`, `!`), comparison operators, and arithmetic. This is the subset of Scala formalized in §3; the implementation checks that predicates stay within this fragment. Importantly, predicates may refer to free variables from the enclosing scope, enabling dependent refinements such as the `b.len == a.len` predicate in the zip signature of Figure 3.

Predicates must be *pure: deterministic and side-effect-free*¹. For instance, a mutable variable cannot appear in a predicate, and equality in predicates is structural, not referential:

```
var x = 3
val y: (Int with y == x) = 3 // rejected because x is mutable
class Box(val value: Int)
val b: (Box with b == Box(3)) = Box(3) // rejected because Box has reference equality
```

We do not currently enforce purity of called functions: our solver assumes all functions are pure, so using an impure function in a predicate is a silent logical error. This unsafe default is a deliberate choice to simplify gradual adoption without imposing a large upfront annotation burden.

A concrete path toward enforcing purity is offered by *capture checking* [48], a new, experimental extension of the Scala 3 compiler.² It tracks *capabilities*: references that confer the authority to perform effects, such as file-system or network access. The types of values and methods record which capabilities they capture or use; a *pure* function is one that captures nothing. Three further extensions—*Stateful Capabilities*, *separation checking* [49], and *safe mode* [29]—together extend this accounting to mutable state: reads and writes are tracked as capability uses, written state is exclusive and un-aliased, and escape hatches such as unchecked casts and reflection are closed, so that no effect or mutation goes untracked.

With these checks in place, purity checking of predicates reduces to a simple condition: reject any predicate that references a capability, calls a method that uses one, or reads a mutable variable. Weaker checks are also conceivable: tracking the *read set* of each predicate (the read-only capabilities it uses) and invalidating established refinement facts whenever an element of that set is written to. Separation checking would make such invalidation sound, as all writes are explicit and un-aliased. Similar ideas underpin refinement type systems for Rust, where ownership likewise guarantees exclusive access to written state (§5). We leave these integrations to future work.

2.3 Non-termination and Partial Correctness

Our system uses a *partial-correctness* semantics: a term e has type $\{x: T \text{ with } p(x)\}$ if, whenever e terminates with a value v , the predicate $p(v)$ holds. No termination guarantee is required for typing. Formally, the term interpretation $\mathcal{E}[[A]]$ (§3.3) asserts that *if* a term evaluates to a value, that value satisfies the type’s interpretation. This is a key difference from prior systems, designed around termination: System FR [16] enforces total correctness and requires termination proofs for all recursive functions, Liquid Haskell requires termination of functions reflected into refinements [44], and F^* [39] and Dafny [27] require specification-level computation to terminate.

Since predicates are ordinary pure terms (§2.2), they may themselves diverge. The termination requirements of the systems above stem from their reading of predicates as formulas in a classical

¹We conjecture that determinism alone is sufficient for soundness: benign side effects (e.g., logging) that do not affect the predicate’s return value would be harmless. We leave formalizing this conjecture to future work.

²The extensions are described in the Scala 3 documentation: <https://nightly.scala-lang.org/docs/reference/experimental/capture-checking/>.

logic, typically that of an SMT solver. We instead define their meaning operationally: $\{x : A \mid p\}$ contains those values of A for which p evaluates to true *whenever it terminates*. This mirrors the semantics of run-time assertions: evaluating p as a dynamic check—as refinement patterns and `.runtimeChecked` do in practice (§2.5)—signals a violation only if p evaluates to false, so a diverging check never fails. Type soundness thus requires no termination assumptions; termination resurfaces only when justifying individual entailment rules. Crucially, a diverging predicate is vacuously satisfied: it behaves like true, and can never be used to prove false.

Termination considerations are confined to a single judgment: the *semantic implication* judgment $\Gamma \models p_1 \Rightarrow p_2$ (§3.3), which the subtyping rule S-REFINE (§3.5) uses to compare the predicates of two refinement types. It reads both sides partially: in every well-formed environment, if p_1 evaluates to true whenever it terminates, then so does p_2 . Thus, no type checking rule needs to refer to termination; it is only needed to justify individual entailment rules, and only some of them. Here and below, a term *terminates* in a given environment if its evaluation produces a value. For example, dropping a trivially true conjunct needs no termination:³

LEMMA 2.1 (AND-TRUE). $\forall \Gamma, t. \Gamma \models (t \ \&\& \ \text{true}) \Rightarrow t$.

Other rules genuinely do:

LEMMA 2.2 (AND-FALSE). $\forall \Gamma, t. t \text{ terminates} \implies \Gamma \models (t \ \&\& \ \text{false}) \Rightarrow \text{false}$.

The termination hypothesis is necessary: writing “diverge” for a closed term that never terminates, we have $\neg(\emptyset \models (\text{diverge} \ \&\& \ \text{false}) \Rightarrow \text{false})$, since the left-hand side diverges and is vacuously satisfied while false is not. The fact that a “morally false” predicate can be vacuously satisfied may look alarming, but it does not impact soundness: a diverging predicate is indistinguishable from true, not from false, and true proves nothing. Writing $\langle \cdot \rangle$ for mutual subtyping, we have:

LEMMA 2.3 (DIVERGING PREDICATES ARE VACUOUS).

- (1) $\forall \Gamma, A. \Gamma \models \{x : A \mid \text{diverge}\} \langle \cdot \rangle \{x : A \mid \text{true}\} \langle \cdot \rangle A$.
- (2) $\forall \Gamma, t, A. t \text{ terminates} \implies \neg(\Gamma \models t : \{x : A \mid \text{false}\})$.
- (3) $\neg(\emptyset \models \{x : \text{Unit} \mid \text{diverge}\} \langle \cdot \rangle \{x : \text{Unit} \mid \text{false}\})$.

A diverging *term*, by contrast, can be given the type $\{x : \text{Unit} \mid \text{false}\}$: the partial-correctness interpretation is vacuous for terms that never produce a value. This is not unsound: by Lemma 2.3(2), no computation that actually completes has this type, so any code path that would exploit such a “proof of false” must first force the diverging computation, and therefore never runs. Unsoundness would arise only if the computation were erased at compile time, allowing code to assume the predicate without running it.

Because the type system is sound without any termination assumption, termination can be handled permissively, as a solver concern rather than a typing one. We therefore give the implication judgment a semantic definition only, and leave developing a syntactic proof system for predicate entailment as future work. In practice, few entailment rules are termination-sensitive: within the checked predicate fragment (§2.2), every construct except function calls trivially terminates, and termination only matters for specific rules, such as the simplification of $t \ \&\& \ \text{false}$ to false above, or the use of a predicate from a called function’s result type. Even with a termination checker in place, proofs would be required only for the specific functions those rules touch—never systematically for every function used in a predicate. We believe this matters for adoption: Scala is a general-purpose language, and systematically requiring termination proofs would create a significant barrier. For now, our solver (§4.3) simply assumes termination of the functions called in predicates,

³The lemmas in this subsection are proved in our Rocq mechanization, in the file `ExamplesPartialSoundness.v`.

as it does purity—a deliberately permissive first approximation. Verifying the termination-sensitive entailment rules and adding a termination checker are future work.

2.4 Mixed-precision Type Inference

The previous sections describe how refinement types are *written* and what they *mean*. This section addresses how expressions actually *receive* refinement types during type checking. A key design goal is backward compatibility: enabling refinement types should not change the inferred types of existing code.

Why not always infer precise types? A natural approach would be to infer refinement types everywhere. For example, systematically give `val x = 1 + 2` its most precise type `{v: Int with v == 1 + 2}` rather than `Int`. This is impractical for three reasons.

First, *backward compatibility*. In Scala, types are not only descriptive—they drive program elaboration. Implicit resolution and overload resolution both depend on the inferred types of arguments. A more precise type can change which implicit instances are found or introduce ambiguities between overloaded methods, breaking code that previously compiled.

Second, *performance*. Carrying precise types for every intermediate expression increases the size of types, leading to higher memory usage and slower type comparisons.

Third, *usability*. Overly detailed inferred types—such as deeply nested unions of singleton types—are harder to read and understand than simple approximations like `Int`.

Our solution: equality facts and selffication. Instead of always inferring precise types, we keep Scala’s existing widening behavior and recover precision on demand through two mechanisms, both standard in refinement type systems [3, 16, 38].

The first mechanism is the *equality fact context*. The T-LET typing rule (Figure 9) records an equality fact $x \sim e_1$ between each binding x and its definition e_1 , where $\sim$ denotes that two terms evaluate to the same value:

$$\frac{\Gamma \vdash e_1 : T_1 \quad \Gamma, x : T_1, x \sim e_1 \vdash e_2 : T_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \text{avoid}(T_2, x)} \quad (\text{T-LET})$$

The binding `val mPlusN = m + n` in Figure 3 is inferred as `Int`, but the equality `mPlusN ~ m + n` is recorded in the context. When `mPlusN` is later used where a refined type is expected, the system can substitute `m + n` for `mPlusN` in predicates and verify the obligation. `avoid` is described in §3.4.

The second mechanism is *selffication* (T-SELF in Figure 9). When checking an expression e of type T against a refinement type, the compiler assigns it the self-referencing type `{x: T with x == e}`, provided e is a valid predicate term. This lifts expressions into types on demand without changing inference for unannotated code.

2.5 Run-time Checks

Refinement predicates are checked statically by default, but when the compiler cannot verify a predicate, two run-time mechanisms are available.

Pattern matching. A refinement type can be used as a pattern, branching on whether a value satisfies the predicate at run time [2]:

```
type ID = {s: String with s.matches(idRegex)}
```

```
"a2e7-e89b" match
  case id: ID => ... // predicate holds, id has type ID
  case _      => ... // predicate does not hold
```

This integrates naturally with Scala's existing pattern-matching infrastructure. Since Scala compiles to the JVM, which erases type parameters at run time, matching against parameterized refinement types such as `List[ID]` is not directly supported, though straightforward workarounds exist (e.g., filtering a collection with the dedicated method).

Checked casts. When the programmer expects a predicate to hold but the solver cannot prove it statically, `.runtimeChecked` [36] performs a dynamic check and throws an exception on failure:

```
val id: ID = "a2e7-e89b".runtimeChecked
// desugars to:
val id: ID =
  if ("a2e7-e89b".matches(idRegex)) "a2e7-e89b".asInstanceOf[ID]
  else throw new IllegalArgumentException()
```

As with other Scala types, `.asInstanceOf` can also be used directly to skip the check entirely, at the programmer's risk. Note that pattern matching on refinement types and `.runtimeChecked` only works for first-order predicates that can be evaluated directly; higher-order values such as functions cannot be checked eagerly, sidestepping the blame assignment problem [21] for now.

3 Metatheory

We prove type soundness for a core calculus that captures the essential features of our design. The calculus extends System $F_{<,>}$ (System $F_{<}$ [5] extended with lower bounds on type variables [1]) with booleans, ints, dependent function types, dependent pairs (sigma types), sum types, union and intersection types, refinement types, loops, and positive equi-recursive types. The mechanization is carried out in Rocq [40] using a fuel-bounded definitional interpreter [1] for the operational semantics, and semantic typing [42] to prove type safety.

To illustrate the scope of the calculus, Figure 4 shows a Scala function that collects elements satisfying a predicate into a refined list, alongside its encoding in the core calculus. The function exercises bounded polymorphism ($S >: T$), higher-order predicates (p), refinement types ($List[\{v : T \mid p(v)\}]$), equi-recursive types (the list itself, encoded as $\mu X. \text{Unit} + (T \times X)$), and tail-recursive iteration (encoded as loop). The mechanization includes this encoding. Running it through the interpreter for example verifies that applying it to $[3, -1, 4, -1, 5]$ with $\lambda x. x > 0$ yields $[3, 4, 5]$.

3.1 Language

The abstract syntax is given in Figure 5. Types and terms are mutually inductive, since refinement types $\{x : A \mid p\}$ embed a term p as a predicate.

Naming convention. The mechanization uses de Bruijn indices throughout. For readability, the paper presentation uses named variables and omits explicit shifts; all rules should be read modulo α -equivalence.

Terms. Bounded type abstractions $\Lambda(X :> L <: U). b$ carry both a lower bound L and an upper bound U , as in $F_{<,>}$, to model Scala's bounded polymorphism.

`loop(a) x. b` is a limited form of general recursion that suffices to express tail recursion: the initial state a is bound to x in the body b , which returns `inl(v1)` to continue with a new state, or `inr(v)` to exit with result v .

Integer arithmetic uses signed 32-bit wrapping semantics (`Int32`) rather than mathematical integers; this matches Scala's runtime model and could in the future serve as a foundation for proving the solver's normalization rules correct. It otherwise has no impact on the formalization.

Scala source

```

enum List[T]:
  case Nil()
  case Cons(head: T, tail: List[T])

@tailrec def collect[T, S >: T](
  xs: List[T],
  p: S => Boolean,
  acc: List[{v: T with p(v)}] =
): List[{v: T with p(v)}] =
  xs match
    case Nil() => acc
    case Cons(x, xs1) =>
      if p(x)
      then collect(xs1, p, Cons(x, acc))
      else collect(xs1, p, acc)

```

Core calculus encoding

```

List[T]  $\triangleq$   $\mu X. \text{Unit} + (T, X)$ 

 $\Lambda(T \text{ :> } \perp <: T). \Lambda(S \text{ :> } T <: T).$ 
 $\lambda xs: \text{List}[T].$ 
 $\lambda p: (S \rightarrow \text{Bool}).$ 
 $\lambda acc: \text{List}[\{v : T \mid p(v)\}].$ 
  loop((xs, acc)) state.
    match state with (rem, cacc)  $\Rightarrow$ 
      match rem with
        inl(_)  $\Rightarrow$  inr(cacc)
      | inr(c)  $\Rightarrow$ 
        match c with (hd, tl)  $\Rightarrow$ 
          if p hd
          then inl((tl, inr((hd, cacc))))
          else inl((tl, cacc))

```

Fig. 4. A collect function in Scala (left) and its encoding in the core calculus (right). The tail-recursive loop becomes a loop combinator whose body returns $\text{inl}(\text{state}')$ to continue or $\text{inr}(\text{result})$ to break. Lists are equi-recursive: $\text{inl}(\text{unit})$ represents the empty list and $\text{inr}((hd, tl))$ a cons cell.

Values. As in other definitional interpreter developments [1, 31], values are distinct from terms: while term lambdas $\lambda x:A. b$ may have free variables, value closures $\langle \rho, \lambda x. b \rangle$ capture their environment ρ as a list of values. This separation reflects the big-step evaluation strategy: evaluation produces values with closed environments, not terms.

Types. The type language includes dependent function types $\Pi x:A. B$, dependent pairs $\Sigma x:A. B$, bounded universal types $\forall (X \text{ :> } L <: U). A$, sum types, union and intersection types, and refinement types $\{x : A \mid p\}$ where the predicate p is an arbitrary term. Refinement types can be nested, as in the implementation.

The base types include singleton types `True` and `False` rather than a single `Bool` because this enables defining the refinement type interpretation succinctly: a value satisfies $\{x : A \mid p\}$ when it belongs to $\mathcal{V} \llbracket A \rrbracket$ and p evaluates to true, which is exactly membership in $\mathcal{E} \llbracket \text{True} \rrbracket$. The conventional `Bool` is recovered as `True` $\vee$ `False`.

The type $\mu X. A$ denotes an equi-recursive type, interpreted as the intersection of all step-indexed approximations (see §3.3). Unlike iso-recursive systems, no explicit fold/unfold terms are needed; instead, the equivalence $\mu X. A \cong A[X \mapsto \mu X. A]$ is established via subtyping rules (§3.5).

Contexts. A typing context Γ (Figure 5) contains three kinds of entries: term bindings $x : A$, type variable bounds $X \text{ :> } L <: U$, and equality facts $a_1 \sim a_2$. Equality facts record that two terms evaluate to the same value. They are introduced by `T-LET`, which records that a let-bound variable equals its definition, and by `T-IF`, which records that the condition equals true or false in the respective branches. These facts are used during subtyping to justify predicate entailment.

3.2 Operational Semantics

The rules for successful evaluation $\rho \vdash a \Downarrow v$ are given in Figure 6. Closures capture the environment at the point of creation: evaluating $\lambda x:A. b$ under ρ yields $\langle \rho, \lambda x. b \rangle$. Application evaluates the

$c := \text{unit} \mid \text{true} \mid \text{false} \mid z \in \mathbb{Z}$	Constant	$v :=$	Value
$op ::= = \mid != \mid < \mid <= \mid >= \mid >$	Operation	c	constant
$\&\& \mid \mid + \mid - \mid * \mid / \mid \%$		(v_1, v_2)	pair
$a, b, f, p :=$	Term	$\text{inl}(v)$	left injection
c	constant	$\text{inr}(v)$	right injection
x	variable	$\langle \rho, \lambda x. b \rangle$	closure
$\lambda x:A. b$	abstraction	$\langle \rho, \Lambda X. b \rangle$	type closure
$f a$	application	$A, B :=$	Type
$\Lambda(X:>L<:U). b$	type abstraction	X	variable
$f [A]$	type application	$\text{Unit} \mid \text{True} \mid \text{False} \mid \text{Int32}$	base types
$\text{let } x:A = a \text{ in } b$	let	$\top$	top
(a_1, a_2)	pair	$\perp$	bottom
$\text{match } a \text{ with } (x, y) \Rightarrow b$	match pair	$\Pi x:A. B$	dependent function
	match sum	$\forall(X:>L<:U). A$	bounded $\forall$
$\text{match } a \text{ with } \text{inl}(x) \Rightarrow b_l \mid \text{inr}(y) \Rightarrow b_r$		$\Sigma x:A. B$	dependent pair
$\text{inl}[A] a$	left injection	$A + B$	sum
$\text{inr}[A] a$	right injection	$\{x : A \mid p\}$	refinement
$a \text{ op } b$	binary operation	$A \vee B$	union
$\text{if } a \text{ then } b_1 \text{ else } b_2$	conditional	$A \wedge B$	intersection
$\text{loop}(a) x. b$	loop	$\mu X. A$	recursive
$\rho :=$	Environment	$\Gamma :=$	Context
$\emptyset$	empty	$\emptyset$	empty
$\rho, x \mapsto v$	binding	$\Gamma, x : A$	term binding
		$\Gamma, X :> L <: U$	type var. bound
		$\Gamma, a_1 \sim a_2$	equality fact

Fig. 5. Abstract syntax of the core calculus. Types and terms are mutually inductive; bindings use de Bruijn indices.

function and argument, then evaluates the body in the closure's environment extended with the argument. In rule E-BINOP, primitive operations are interpreted by the partial function $\text{evalop}(op, v_a, v_b)$ (eval_bin_op in the mechanization), which is undefined for ill-typed combinations of operands, making such applications stuck.

Erased semantics. Types are erased at runtime, so they have no operational semantics. This is reflected in the evaluation rules for type abstractions and applications, which simply ignore the type arguments. This matches Scala's runtime model, where types are erased on the JVM.

Definitional interpreter. In the mechanization, evaluation is implemented as a fuel-bounded definitional interpreter (Figure 7), following Amin and Rompf [1]. The interpreter is a recursive Rocq function that returns a layered option type: `None` for timeout (fuel exhausted), `Some None` for a stuck state, or `Some (Some v)` for successful evaluation to a value v . We write $\rho \vdash a \Downarrow^? r$

Big-step evaluation rules

$$\boxed{\rho \vdash a \Downarrow v}$$

$\rho \vdash c \Downarrow c$	(E-CONST)	$\rho \vdash a \Downarrow v_a \quad \rho \vdash b \Downarrow v_b$ $\text{evalop}(op, v_a, v_b) = r$	(E-BINOP)
$\frac{x \mapsto v \in \rho}{\rho \vdash x \Downarrow v}$	(E-VAR)	$\rho \vdash a \Downarrow \text{true} \quad \rho \vdash b_1 \Downarrow v$ $\rho \vdash \text{if } a \text{ then } b_1 \text{ else } b_2 \Downarrow v$	(E-IFTRUE)
$\rho \vdash \lambda x:A. b \Downarrow \langle \rho, \lambda x. b \rangle$	(E-ABS)	$\rho \vdash a \Downarrow \text{false} \quad \rho \vdash b_2 \Downarrow v$ $\rho \vdash \text{if } a \text{ then } b_1 \text{ else } b_2 \Downarrow v$	(E-IFFALSE)
$\rho \vdash \Lambda(X:>L<:U). b \Downarrow \langle \rho, \Lambda X. b \rangle$	(E-TABS)	$\rho \vdash a \Downarrow v$ $\rho \vdash \text{inl}[B] a \Downarrow \text{inl}(v)$	(E-INL)
$\frac{\rho \vdash f \Downarrow \langle \rho_f, \lambda x. b \rangle \quad \rho \vdash a \Downarrow v_a \quad \rho_f, x \mapsto v_a \vdash b \Downarrow v}{\rho \vdash f a \Downarrow v}$	(E-APP)	$\rho \vdash a \Downarrow v$ $\rho \vdash \text{inr}[A] a \Downarrow \text{inr}(v)$	(E-INR)
$\frac{\rho \vdash f \Downarrow \langle \rho_f, \Lambda X. b \rangle \quad \rho_f \vdash b \Downarrow v}{\rho \vdash f [A] \Downarrow v}$	(E-TAPP)	$\rho \vdash a \Downarrow v_0$ $\rho, x \mapsto v_0 \vdash b \Downarrow \text{inr}(v)$ $\rho \vdash \text{loop}(a) x. b \Downarrow v$	(E-LOOPEXIT)
$\frac{\rho \vdash a \Downarrow v_a \quad \rho, x \mapsto v_a \vdash b \Downarrow v}{\rho \vdash \text{let } x:A = a \text{ in } b \Downarrow v}$	(E-LET)	$\rho \vdash a \Downarrow v_0 \quad \rho, x \mapsto v_0 \vdash b \Downarrow \text{inl}(v_1)$ $\rho \vdash \text{loop}(v_1) x. b \Downarrow v$	(E-LOOPCONT)
$\frac{\rho \vdash a_1 \Downarrow v_1 \quad \rho \vdash a_2 \Downarrow v_2}{\rho \vdash (a_1, a_2) \Downarrow (v_1, v_2)}$	(E-PAIR)	$\rho \vdash a \Downarrow (v_1, v_2) \quad \rho, x \mapsto v_1, y \mapsto v_2 \vdash b \Downarrow v$ $\rho \vdash \text{match } a \text{ with } (x, y) \Rightarrow b \Downarrow v$	(E-MATCHPAIR)
		$\rho \vdash a \Downarrow \text{inl}(v) \quad \rho, x \mapsto v \vdash b_l \Downarrow w$ $\rho \vdash \text{match } a \text{ with } \text{inl}(x) \Rightarrow b_l \mid \text{inr}(y) \Rightarrow b_r \Downarrow w$	(E-MATCHSUML)
		$\rho \vdash a \Downarrow \text{inr}(v) \quad \rho, y \mapsto v \vdash b_r \Downarrow w$ $\rho \vdash \text{match } a \text{ with } \text{inl}(x) \Rightarrow b_l \mid \text{inr}(y) \Rightarrow b_r \Downarrow w$	(E-MATCHSUMR)

Fig. 6. Successful evaluation rules ($\rho \vdash a \Downarrow v$), derived from the definitional interpreter (Figure 7).

when evaluation terminates with result *Some* r (either stuck or a value), and $\rho \vdash a \Downarrow v$ for the special case where $r = \text{Some } v$. The big-step rules in Figure 6 show only successful evaluation; the mechanization accounts for all three outcomes, as shown in Figure 7.

Fuel bounds depth, not steps. The fuel parameter bounds the *depth* of evaluation, not the total number of steps. When evaluating an application, for example, the argument, the function, and the body are all evaluated with the same, decremented fuel. This follows Amin and Rompf [1], but differs from approaches that decrement fuel at every reduction step [31, 32]: for an application $f a$, those approaches partition the available fuel across the sub-evaluations ($\text{fuel}_f + \text{fuel}_a + \text{fuel}_{\text{body}} \approx \text{fuel}$), whereas ours passes the same decremented fuel to each ($\text{fuel}_f = \text{fuel}_a = \text{fuel}_{\text{body}} = \text{fuel} - 1$). The advantage is simplicity: depth-bounded fuel gives a natural termination measure for structural induction. The disadvantage is that it does not provide a global step count that can be correlated

Definitional interpreter (excerpt) $\text{eval } n \ \rho \ a$

```

Fixpoint eval (fuel: nat) (env: list Value) (t: Term)
  : option (option Value) :=
  match fuel with
  | 0 => None (* timeout *)
  | S fuel' =>
    match t with
    | tabs _ b => Some (Some (vabs env b))
    | tapp f a =>
      match eval fuel' env f with
      | None => None
      | Some (Some (vabs envf b)) =>
        match eval fuel' env a with
        | Some (Some va) => eval fuel' (va :: envf) b
        | o => o (* propagate timeout or stuck *)
      end
    | _ => Some None (* stuck *)
    end
  ...
end.

```

Termination $\rho \vdash a \Downarrow^? r$

$$\rho \vdash a \Downarrow^? r \triangleq \exists n. \text{eval } n \ \rho \ a = \text{Some } r$$

Successful evaluation $\rho \vdash a \Downarrow v$

$$\rho \vdash a \Downarrow v \triangleq \rho \vdash a \Downarrow^? \text{Some } v$$

Fig. 7. Fuel-bounded definitional interpreter, following Amin and Rompf [1]. The interpreter returns **None** (timeout), **Some None** (stuck), or **Some (Some v)** (success).

with recursive-type unfoldings, a technique commonly used to prove soundness of equi-recursive types via step-indexed logical relations [42]. We side-step this limitation by making our value interpretation step-index-free (§3.3).

3.3 Interpretation

The semantic framework is given in Figure 8. We describe each component below.

Value interpretation. The semantic approach to type soundness represents types as predicates on values: a semantic type is a function $\text{Value} \rightarrow \text{Prop}$. The value interpretation $\mathcal{V}\llbracket A \rrbracket_\delta^\rho$ maps a syntactic type A to a semantic type, parameterized by a type variable assignment δ (mapping type variables to semantic types) and a term variable assignment ρ (mapping term variables to values). The type variable assignment enables the interpretation of polymorphic types; the term variable assignment enables the interpretation of dependent types and refinement predicates. Using an

explicit term variable assignment rather than a closing substitution is less standard, but is a natural fit for our environment-passing definitional interpreter.

Base types are interpreted as expected: $\mathcal{V}[\text{Unit}]$ accepts only unit, $\mathcal{V}[\text{True}]$ and $\mathcal{V}[\text{False}]$ accept the corresponding boolean values, and $\mathcal{V}[\text{Int32}]$ accepts any integer literal. A value satisfies a function type $\Pi x:A. B$ if it is a closure $\langle \rho_f, \lambda x. b \rangle$ such that for every argument v_a satisfying $\mathcal{V}[A]$, the body b evaluated in the closure's environment ρ_f extended with v_a satisfies $\mathcal{E}[B]$. Note that the body is evaluated in the closure's captured environment, not the caller's. A refinement type $\{x : A \mid p\}$ conjoins membership in $\mathcal{V}[A]$ with the predicate p belonging to $\mathcal{E}[\text{True}]$ —i.e., evaluating to true whenever it terminates. Union and intersection types are interpreted as set-theoretic union and intersection, respectively.

The recursive type $\mu X. A$ is interpreted as the intersection of all finite approximations $\forall n. F^n(v)$, where $F^0(v) = \top$ and $F^{n+1}(v) = \mathcal{V}[A]_{\delta[X \mapsto F^n]}^\rho(v)$. Intuitively, F^n captures the behavior of $\mu X. A$ up to n unfoldings. A value inhabits the recursive type only if it satisfies all approximations. This interpretation is step-index-free in the sense that the final semantic type $\mathcal{V}[\mu X. A]$ universally quantifies over all approximation levels n , so it does not depend on any external step counter. The step indexing is confined to the internal construction of the approximation sequence.

Term interpretation. The term interpretation $\mathcal{E}[A]_\delta^\rho(a)$ lifts the value interpretation to a predicate on terms. As described in §2.3, a term a has type A if, whenever evaluation of a terminates with a result r (i.e., $\rho \vdash a \Downarrow^? r$), then r must be a value v satisfying $\mathcal{V}[A]_\delta^\rho$. This rules out stuck states: if evaluation completes, it must produce a well-typed value. The statement is vacuously true for diverging terms, making this a partial correctness property.

Context well-formedness. The well-formedness predicate $\text{wf}(\delta, \Gamma, \rho)$ requires that the assignments δ and ρ are consistent with the context Γ . Concretely, it requires three conditions: (1) for every term binding $x : A$ in Γ , the value $\rho(x)$ satisfies $\mathcal{V}[A]_\delta^\rho$; (2) for every type variable bound $X :> L <: U$ in Γ , the semantic type $\delta(X)$ lies between $\mathcal{V}[L]$ and $\mathcal{V}[U]$; and (3) for every equality fact $a_1 \sim a_2$ in Γ , there exists a value v such that both a_1 and a_2 evaluate to v . Condition (3) is slightly simplified here: in the mechanization, each side of a fact carries a depth tag recording how many bindings were in scope when the fact was introduced, and the term is evaluated in the corresponding suffix of the environment. The simpler statement using the full environment can be recovered up to captured environments in closures: evaluating a term in a longer environment produces the same result, except that closures capture the environment available at their creation point. This is the same subtlety addressed by the evaluation weakening lemma (Lemma 3.7).

Semantic typing. The semantic typing judgment $\Gamma \models a : A$ universally quantifies over all type variable assignments δ and term variable assignments ρ that are well-formed with respect to Γ , and requires that $\mathcal{E}[A]_\delta^\rho(a)$ holds. In other words, in any environment consistent with the context, whenever evaluation of a terminates, the result is a value satisfying the type's interpretation.

Semantic subtyping. The semantic subtyping judgment $\Gamma \models A <: B$ similarly quantifies over all well-formed assignments and requires $\mathcal{V}[A]_\delta^\rho \subseteq \mathcal{V}[B]_\delta^\rho$, where semantic type inclusion $S_1 \subseteq S_2$ is defined as pointwise implication: $\forall v. S_1(v) \implies S_2(v)$. In other words, every value that satisfies A also satisfies B .

Semantic implication. The semantic implication judgment $\Gamma \models p_1 \implies p_2$ requires that, in any well-formed environment, if p_1 evaluates to true (i.e., $\mathcal{E}[\text{True}](p_1)$ holds), then so does p_2 . This judgment is used in the refinement subtyping rule S-REFINE to compare predicates.

Value interpretation

$$\boxed{\mathcal{V}[[A]]_\delta^\rho(v)}$$

$$\begin{aligned}
\mathcal{V}[[X]]_\delta^\rho(v) &\triangleq \delta(X)(v) \\
\mathcal{V}[[\text{Unit}]]_\delta^\rho(v) &\triangleq v = \text{unit} \\
\mathcal{V}[[\text{True}]]_\delta^\rho(v) &\triangleq v = \text{true} \\
\mathcal{V}[[\text{False}]]_\delta^\rho(v) &\triangleq v = \text{false} \\
\mathcal{V}[[\text{Int32}]]_\delta^\rho(v) &\triangleq \exists z. v = z \\
\mathcal{V}[[\top]]_\delta^\rho(v) &\triangleq \top \\
\mathcal{V}[[\perp]]_\delta^\rho(v) &\triangleq \perp \\
\mathcal{V}[[\Pi x:A. B]]_\delta^\rho(v) &\triangleq \exists \rho_f, b. v = \langle \rho_f, \lambda x. b \rangle \wedge \forall v_a. \mathcal{V}[[A]]_\delta^\rho(v_a) \implies \mathcal{E}[[B]]_\delta^{\rho_f[x \mapsto v_a]}(b) \\
\mathcal{V}[[\forall (X:>L<:U). B]]_\delta^\rho(v) &\triangleq \exists \rho_f, b. v = \langle \rho_f, \lambda X. b \rangle \wedge \forall S. \mathcal{V}[[L]]_\delta^\rho \subseteq S \subseteq \mathcal{V}[[U]]_\delta^\rho \implies \mathcal{E}[[B]]_\delta^{\rho_f[X \mapsto S]}(b) \\
\mathcal{V}[[\Sigma x:A. B]]_\delta^\rho(v) &\triangleq \exists v_1, v_2. v = (v_1, v_2) \wedge \mathcal{V}[[A]]_\delta^\rho(v_1) \wedge \mathcal{V}[[B]]_\delta^{\rho[x \mapsto v_1]}(v_2) \\
\mathcal{V}[[A + B]]_\delta^\rho(v) &\triangleq (\exists w. v = \text{inl}(w) \wedge \mathcal{V}[[A]]_\delta^\rho(w)) \vee (\exists w. v = \text{inr}(w) \wedge \mathcal{V}[[B]]_\delta^\rho(w)) \\
\mathcal{V}[[\{x : A \mid p\}]]_\delta^\rho(v) &\triangleq \mathcal{V}[[A]]_\delta^\rho(v) \wedge \mathcal{E}[[\text{True}]]_\delta^{\rho[x \mapsto v]}(p) \\
\mathcal{V}[[A \vee B]]_\delta^\rho(v) &\triangleq \mathcal{V}[[A]]_\delta^\rho(v) \vee \mathcal{V}[[B]]_\delta^\rho(v) \\
\mathcal{V}[[A \wedge B]]_\delta^\rho(v) &\triangleq \mathcal{V}[[A]]_\delta^\rho(v) \wedge \mathcal{V}[[B]]_\delta^\rho(v) \\
\mathcal{V}[[\mu X. A]]_\delta^\rho(v) &\triangleq \forall n. F^n(v) \quad \text{where } F^0(v) = \top, F^{n+1}(v) = \mathcal{V}[[A]]_\delta^{\rho[X \mapsto F^n]}(v)
\end{aligned}$$

Term interpretation

$$\boxed{\mathcal{E}[[A]]_\delta^\rho(a)}$$

$$\mathcal{E}[[A]]_\delta^\rho(a) \triangleq \forall r. \rho \vdash a \Downarrow^? r \implies \exists v. r = \text{Some } v \wedge \mathcal{V}[[A]]_\delta^\rho(v)$$

Context well-formedness

$$\boxed{\text{wf}(\delta, \Gamma, \rho)}$$

$$\begin{aligned}
\text{wf}(\delta, \Gamma, \rho) &\triangleq \forall (x : A) \in \Gamma. \mathcal{V}[[A]]_\delta^\rho(\rho(x)) \\
&\quad \wedge \forall (X:>L<:U) \in \Gamma. \mathcal{V}[[L]]_\delta^\rho \subseteq \delta(X) \subseteq \mathcal{V}[[U]]_\delta^\rho \\
&\quad \wedge \forall (a_1 \sim a_2) \in \Gamma. \exists v. \rho \vdash a_1 \Downarrow v \wedge \rho \vdash a_2 \Downarrow v
\end{aligned}$$

Semantic typing

$$\boxed{\Gamma \models a : A}$$

$$\Gamma \models a : A \triangleq \forall \delta, \rho. \text{wf}(\delta, \Gamma, \rho) \implies \mathcal{E}[[A]]_\delta^\rho(a)$$

Semantic subtyping

$$\boxed{\Gamma \models A <: B}$$

$$\Gamma \models A <: B \triangleq \forall \delta, \rho. \text{wf}(\delta, \Gamma, \rho) \implies \mathcal{V}[[A]]_\delta^\rho \subseteq \mathcal{V}[[B]]_\delta^\rho$$

Semantic implication

$$\boxed{\Gamma \models p_1 \Rightarrow p_2}$$

$$\Gamma \models p_1 \Rightarrow p_2 \triangleq \forall \delta, \rho. \text{wf}(\delta, \Gamma, \rho) \implies \mathcal{E}[[\text{True}]]_\delta^\rho(p_1) \implies \mathcal{E}[[\text{True}]]_\delta^\rho(p_2)$$

Semantic type inclusion

$$\boxed{S_1 \subseteq S_2}$$

$$S_1 \subseteq S_2 \triangleq \forall v. S_1(v) \implies S_2(v)$$

Fig. 8. Semantic framework. The value interpretation $\mathcal{V}[[\cdot]]$ maps syntactic types to predicates on values. The term interpretation $\mathcal{E}[[\cdot]]$ lifts this to a partial correctness assertion on terms. Semantic typing and subtyping quantify over all well-formed environments.

3.4 Typing

The semantic typing judgment $\Gamma \models a : A$ (Figure 8) requires that, in any well-formed environment, $\mathcal{E}[A]$ holds for a —i.e., whenever evaluation terminates, the result is a value in $\mathcal{V}[A]$. The typing rules in Figure 9 are proven sound with respect to this judgment. Each rule corresponds to a proven lemma. Noteworthy rules include:

- **T-LET** records the equality $x \sim a$ in the context, enabling the body to use facts about the bound variable's value. The result type uses $\text{avoid}(B, x)$ to remove references to the bound variable, which goes out of scope.
- **T-SELF** (selfification) refines a first-order value to a singleton type $\{x : A \mid x == a\}$. The $\text{firstorder}(A)$ check ensures that A is a type for which equality comparison is defined—concretely, `Unit`, `True`, `False`, `Int32`, or a refinement of one of these. Functions, closures, and polymorphic values are excluded because they do not support runtime equality.
- **T-IF** adds equality facts $a \sim \text{true}$ and $a \sim \text{false}$ in the respective branches, and returns a union type $B_1 \vee B_2$ of the two branches' types.
- **T-LOOP** types a loop whose body returns $A + B$: `inl` continues with a new accumulator of type A , `inr` exits with a result of type B . This rule allows recursion without typing a self-reference, avoiding the need for step-indexed reasoning that would typically be required for a general fixpoint combinator.

ANF restriction. Several typing rules require certain subterms to be variables rather than arbitrary expressions. In **T-APP**, the argument must be a variable y , so that the substitution $B[x \mapsto y]$ in the result type remains a syntactic type (substituting an arbitrary term could produce a type containing a non-variable subterm, complicating the interpretation). Similarly, **T-PAIR** requires the first component to be a variable. This administrative normal form (ANF) restriction does not reduce expressiveness: any argument expression can be let-bound first, with the let-bound variable used as the argument. This is exactly what the implementation does via skolems (§4.2), which hoist arguments to local bindings so that dependent result types can refer to them by name.

Avoidance. The $\text{avoid}(A, x)$ operation removes occurrences of a bound variable x from a type A , producing a well-scoped result type. It traverses the type structurally, guided by a polarity flag that flips at contravariant positions (the domain of Π and the upper bound of $\forall$). When a refinement predicate mentions x , it is replaced by `true` in positive polarity (yielding a supertype) or `false` in negative polarity (yielding a subtype). For recursive types $\mu Y. B$, avoidance recurses into the body only when $\text{spos}(Y, B)$ holds; otherwise it falls back to $\top$ or $\perp$. The soundness of this operation is established in Lemma 3.8.

3.5 Subtyping

The semantic subtyping judgment $\Gamma \models A <: B$ (Figure 8) requires that, in any well-formed environment, $\mathcal{V}[A] \subseteq \mathcal{V}[B]$. The subtyping rules in Figure 10 are proven sound with respect to this judgment. Function types are contravariant in their domain and covariant in their codomain (**S-FUN**); bounded universal types are covariant in the lower bound, contravariant in the upper bound, and covariant in the body (**S-FORALL**). Refinement types support two rules: **S-REFINEBASE** drops the predicate (every refined type is a subtype of its base), and **S-REFINE** compares both base types and predicates, using the semantic implication judgment $\Gamma \models p_1 \Rightarrow p_2$ to establish that the predicate of the subtype entails the predicate of the supertype.

The equi-recursive rules **S-MU-FOLD** and **S-MU-UNFOLD** establish the isomorphism $\mu X. A \cong A[X \mapsto \mu X. A]$, requiring that X appears only in strictly positive positions in A , as defined by the

Semantic typing rules

			$\boxed{\Gamma \models a : A}$
$\Gamma \models \text{unit} : \text{Unit}$	(T-UNIT)	$\frac{\Gamma \models y : A \quad \Gamma \models a_2 : B[x \mapsto y]}{\Gamma \models (y, a_2) : \Sigma x:A. B}$	(T-PAIR)
$\Gamma \models \text{true} : \text{True}$	(T-TRUE)		
$\Gamma \models \text{false} : \text{False}$	(T-FALSE)	$\frac{\Gamma \models a : A \quad \Gamma \models b : A \quad \text{compat}(op, A)}{\Gamma \models a \text{ op } b : \text{result}(op, A)}$	(T-BINOP)
$\Gamma \models z : \text{Int32}$	(T-INT)		
$\frac{x : A \in \Gamma}{\Gamma \models x : A}$	(T-VAR)	$\frac{\Gamma \models a : \text{True} \vee \text{False} \quad \Gamma, a \sim \text{true} \models b_1 : B_1 \quad \Gamma, a \sim \text{false} \models b_2 : B_2}{\Gamma \models \text{if } a \text{ then } b_1 \text{ else } b_2 : B_1 \vee B_2}$	(T-IF)
$\frac{\Gamma, x : A \models b : B}{\Gamma \models \lambda x:A. b : \Pi x:A. B}$	(T-ABS)	$\frac{\Gamma \models a : A}{\Gamma \models \text{inl}[B] a : A + B}$	(T-INL)
$\frac{\Gamma \models f : \Pi x:A. B \quad \Gamma \models y : A}{\Gamma \models f y : B[x \mapsto y]}$	(T-APP)	$\frac{\Gamma \models a : B}{\Gamma \models \text{inr}[A] a : A + B}$	(T-INR)
$\frac{\Gamma, X :> L <: U \models b : B}{\Gamma \models \Lambda(X:>L<:U). b : \forall(X:>L<:U). B}$	(T-TABS)	$\frac{\Gamma \models a : A \quad \Gamma, x : A \models b : A + B}{\Gamma \models \text{loop}(a) x. b : B}$	(T-LOOP)
$\frac{\Gamma \models f : \forall(X:>L<:U). B \quad \Gamma \models L <: A \quad \Gamma \models A <: U}{\Gamma \models f[A] : B[X \mapsto A]}$	(T-TAPP)	$\frac{\Gamma \models a : A \quad \text{firstorder}(A)}{\Gamma \models a : \{x : A \mid x == a\}}$	(T-SELF)
$\frac{\Gamma \models a : A \quad \Gamma, x : A, x \sim a \models b : B}{\Gamma \models \text{let } x:A = a \text{ in } b : \text{avoid}(B, x)}$	(T-LET)	$\frac{\Gamma \models a : A \quad \Gamma \models A <: B}{\Gamma \models a : B}$	(T-SUB)
$\frac{\Gamma \models a : \Sigma x:A. B \quad \Gamma, x : A, y : B \models b : C}{\Gamma \models \text{match } a \text{ with } (x, y) \Rightarrow b : \text{avoid}(\text{avoid}(C, y), x)}$			(T-MATCHPAIR)
$\frac{\Gamma \models a : A_1 + A_2 \quad \Gamma, x : A_1, a \sim \text{inl}(x) \models b_1 : B_1 \quad \Gamma, y : A_2, a \sim \text{inr}(y) \models b_2 : B_2}{\Gamma \models \text{match } a \text{ with } \text{inl}(x) \Rightarrow b_1 \mid \text{inr}(y) \Rightarrow b_2 : \text{avoid}(B_1, x) \vee \text{avoid}(B_2, y)}$			(T-MATCHSUM)

Fig. 9. Typing rules. All rules are proven sound with respect to the semantic typing judgment.

predicate $\text{spos}(X, A)$ (Figure 11). Strict positivity means X never appears to the left of a function arrow or in the bounds of a universal type. For union types, spos requires X to be entirely absent; this stronger condition is needed for a distributing lemma used in the fold/unfold proofs. Nested recursive types $\mu Y. B$ require both $\text{spos}(Y, B)$ and $X \notin B$: the latter prevents the outer variable from leaking into the inner fixpoint's approximation sequence.

3.6 Soundness and Main Lemmas

Each typing rule in Figure 9 and each subtyping rule in Figure 10 is individually proven sound as a lemma about the semantic judgments $\Gamma \models a : A$ and $\Gamma \models A <: B$. Since these semantic judgments directly assert that well-typed terms do not get stuck and that subtyping preserves value

Semantic subtyping rules $\Gamma \models A <: B$

$\Gamma \models A <: A$	(S-REFL)	$\Gamma \models A \wedge B <: B$	(S-ANDR)
$\frac{\Gamma \models A <: B \quad \Gamma \models B <: C}{\Gamma \models A <: C}$	(S-TRANS)	$\frac{\Gamma \models C <: A \quad \Gamma \models C <: B}{\Gamma \models C <: A \wedge B}$	(S-AND)
$\Gamma \models A <: \top$	(S-TOP)	$\Gamma \models \{x : A \mid p\} <: A$	(S-REFINEBASE)
$\Gamma \models \perp <: A$	(S-BOT)	$\frac{\Gamma \models A <: B \quad \Gamma, x : A \models p_1 \Rightarrow p_2}{\Gamma \models \{x : A \mid p_1\} <: \{x : B \mid p_2\}}$	(S-REFINE)
$\frac{\Gamma \models B_1 <: A_1 \quad \Gamma, x : A_1 \models A_2 <: B_2}{\Gamma \models \Pi x:A_1. A_2 <: \Pi x:B_1. B_2}$	(S-FUN)	$\frac{X := L <: U \in \Gamma}{\Gamma \models X <: U}$	(S-TVAR-UPPER)
$\frac{\Gamma \models A_1 <: B_1 \quad \Gamma, x : A_1 \models A_2 <: B_2}{\Gamma \models \Sigma x:A_1. A_2 <: \Sigma x:B_1. B_2}$	(S-SIGMA)	$\frac{X := L <: U \in \Gamma}{\Gamma \models L <: X}$	(S-TVAR-LOWER)
$\Gamma \models A <: A \vee B$	(S-ORL)	$\frac{\text{spos}(X, A)}{\Gamma \models \mu X. A <: A[X \mapsto \mu X. A]}$	(S-MU-UNFOLD)
$\Gamma \models B <: A \vee B$	(S-ORR)	$\frac{\text{spos}(X, A)}{\Gamma \models A[X \mapsto \mu X. A] <: \mu X. A}$	(S-MU-FOLD)
$\frac{\Gamma \models A <: C \quad \Gamma \models B <: C}{\Gamma \models A \vee B <: C}$	(S-OR)		
$\Gamma \models A \wedge B <: A$	(S-ANDL)		
$\frac{\Gamma \models L_1 <: L_2 \quad \Gamma \models U_2 <: U_1 \quad \Gamma, X := L_2 <: U_2 \models A <: B}{\Gamma \models \forall (X := L_1 <: U_1). A <: \forall (X := L_2 <: U_2). B}$			(S-FORALL)

Fig. 10. Subtyping rules. All rules are proven sound with respect to the semantic subtyping judgment.

Strict positivity $\text{spos}(X, A)$

$$\begin{aligned}
&\text{spos}(X, A) \triangleq \text{True} \quad \text{if } A \text{ is a base type or type variable} \\
&\text{spos}(X, \Pi x:A. B) \triangleq X \notin A \wedge \text{spos}(X, B) \\
&\text{spos}(X, \forall (Y := L <: U). B) \triangleq X \notin L \wedge X \notin U \wedge \text{spos}(X, B) \\
&\text{spos}(X, \mu Y. A) \triangleq \text{spos}(Y, A) \wedge X \notin A \\
&\text{spos}(X, \Sigma x:A. B) \triangleq \text{spos}(X, A) \wedge \text{spos}(X, B) \\
&\text{spos}(X, A + B) \triangleq \text{spos}(X, A) \wedge \text{spos}(X, B) \\
&\text{spos}(X, \{x : A \mid p\}) \triangleq \text{spos}(X, A) \\
&\text{spos}(X, A \vee B) \triangleq X \notin A \wedge X \notin B \\
&\text{spos}(X, A \wedge B) \triangleq \text{spos}(X, A) \wedge \text{spos}(X, B)
\end{aligned}$$

Fig. 11. Strict positivity. $\text{spos}(X, A)$ holds when X appears only in strictly positive positions in A : never to the left of a function arrow, and never in the bounds of a universal type. Union types require X to be entirely absent (needed for the distributing lemma).

membership, any derivation composed from these rules inherits the safety guarantee. This is the fundamental theorem of the logical relation.

Adequacy. To provide a conventional syntactic interface, we also define inductive syntactic typing and subtyping judgments that mirror the semantic rules of Figures 9 and 10. The adequacy theorems show that these syntactic judgments imply their semantic counterparts. The proofs proceed by straightforward induction on the derivation, applying the corresponding semantic lemma at each constructor.

THEOREM 3.1 (ADEQUACY OF TYPING). *If $\Gamma \vdash a : A$ then $\Gamma \models a : A$.*

THEOREM 3.2 (ADEQUACY OF SUBTYPING). *If $\Gamma \vdash A <: B$ then $\Gamma \models A <: B$.*

Note that the semantic implication judgment $\Gamma \models p_1 \Rightarrow p_2$ has no syntactic counterpart: we do not define inference rules for it. Instead, it remains an entirely semantic notion, used directly in the subtyping rule S-REFINE. This reflects the design of the implementation, where predicate entailment is delegated to the solver (§4.3) rather than derived from syntactic rules. Syntactic entailment rules could nonetheless be proved sound against the semantic judgment, just as the typing and subtyping rules are; Lemmas 2.1 and 2.2 (§2.3) exemplify this, showing that some rules hold unconditionally while others require termination assumptions.

This separation between syntactic and semantic judgments allows us to state and use typing rules in the familiar inductive style, while the soundness guarantee flows from the semantic interpretation.

Weakening and substitution for the interpretation. The semantic typing and subtyping proofs require that the value interpretation is well-behaved under changes to the environment. The key lemmas establish that the interpretation depends only on the variables the type mentions: extending the environment with a fresh binding leaves it unchanged, and a variable can be substituted away when the environment determines its value. We write $\text{FV}(A)$ for the free (term and type) variables of A .⁴

LEMMA 3.3 (TERM WEAKENING). *If $x \notin \text{FV}(A)$, then $\mathcal{V}[A]_\delta^\rho = \mathcal{V}[A]_\delta^{\rho[x \mapsto v]}$.*

LEMMA 3.4 (TERM SUBSTITUTION). *If $\rho(y) = v$, then $\mathcal{V}[A]_\delta^{\rho[x \mapsto v]} = \mathcal{V}[A[x \mapsto y]]_\delta^\rho$.*

LEMMA 3.5 (TYPE WEAKENING). *If $X \notin \text{FV}(A)$, then $\mathcal{V}[A]_\delta^\rho = \mathcal{V}[A]_{\delta[X \mapsto S]}^\rho$.*

LEMMA 3.6 (TYPE SUBSTITUTION). $\mathcal{V}[B]_{\delta[X \mapsto \mathcal{V}[A]_\delta^\rho]}^\rho = \mathcal{V}[B[X \mapsto A]]_\delta^\rho$

The mechanization proves more general versions for multi-element extensions, from which these are derived. These lemmas are used pervasively—for instance, term substitution is needed for T-APP and T-LET, and type substitution for T-TAPP and S-MU-FOLD/S-MU-UNFOLD.

Weakening and substitution for evaluation. The interpretation lemmas above reduce questions about the semantic types to questions about evaluation under modified environments. These in turn require corresponding lemmas about the evaluator. The difficulty is that evaluation results are not preserved exactly: when the environment is extended, closures capture the larger environment, producing structurally different values. To handle this, we define a *compatibility relation* $r_1 \approx r_2$ on results that relates first-order values to themselves and closures to closures with correspondingly extended captured environments. The evaluation weakening lemmas then take the following simulation-style form:

⁴The mechanization states these lemmas in de Bruijn form, where extending the environment corresponds to an explicit index shift on the type and the freshness side conditions are implicit.

LEMMA 3.7 (EVALUATION WEAKENING). *Suppose $x \notin \text{FV}(a)$. If $\rho \vdash a \Downarrow^? r$, then there exists $r' \approx r$ such that $\rho[x \mapsto v] \vdash a \Downarrow^? r'$. Conversely, if $\rho[x \mapsto v] \vdash a \Downarrow^? r$, then there exists $r' \approx r$ such that $\rho \vdash a \Downarrow^? r'$.*

This simulation-style argument is needed because the definitional interpreter uses closures rather than substitution: evaluating a lambda under a longer environment produces a closure capturing that longer environment. Analogous lemmas handle term substitution, with a corresponding substitution compatibility relation.

Avoidance. The avoid operation (§3.4) must preserve the subtyping relationship in the appropriate direction. The key lemma establishes this for both polarities simultaneously:

LEMMA 3.8 (AVOIDANCE SOUNDNESS). *For all v :*

- (1) $\mathcal{V}\llbracket A \rrbracket_{\delta}^{\rho}(v) \implies \mathcal{V}\llbracket \text{avoid}(A, x)^+ \rrbracket_{\delta}^{\rho}(v)$ (positive: supertype)
- (2) $\mathcal{V}\llbracket \text{avoid}(A, x)^- \rrbracket_{\delta}^{\rho}(v) \implies \mathcal{V}\llbracket A \rrbracket_{\delta}^{\rho}(v)$ (negative: subtype)

These are proven together by induction on A , flipping polarity at contravariant positions. The $\text{avoid}(B, x)$ appearing in the typing rules (e.g., T-LET) uses positive polarity, $\text{avoid}(B, x)^+$, producing a supertype of B .

Positivity and fold/unfold. The fold and unfold rules for $\mu X. A$ require showing that the interpretation of the recursive type coincides with its one-step unfolding: $\mathcal{V}\llbracket \mu X. A \rrbracket = \mathcal{V}\llbracket A[X \mapsto \mu X. A] \rrbracket$. The key ingredients, adapted from Hamza et al. [16], are:

LEMMA 3.9 (MONOTONICITY). *If $\text{spos}(X, A)$ and $S_1 \subseteq S_2$, then $\mathcal{V}\llbracket A \rrbracket_{\delta[X \mapsto S_1]} \subseteq \mathcal{V}\llbracket A \rrbracket_{\delta[X \mapsto S_2]}$.*

LEMMA 3.10 (DISTRIBUTION). *Let $S_{\cap} = \lambda w. \forall n. S_n(w)$. If $\text{spos}(X, A)$ and $\forall n. \mathcal{V}\llbracket A \rrbracket_{\delta[X \mapsto S_n]}(v)$, then $\mathcal{V}\llbracket A \rrbracket_{\delta[X \mapsto S_{\cap}]}(v)$.*

Monotonicity says that the interpretation of a strictly positive type is monotone in the type variable's denotation. Distribution says that an intersection over a family of denotations can be pushed inside the interpretation. Together, these establish that the approximation sequence $F^0 \supseteq F^1 \supseteq \dots$ converges to a fixpoint, so $\forall n. F^n(v)$ is equivalent to $F(\lambda w. \forall n. F^n(w))(v)$, which is exactly $\mathcal{V}\llbracket A[X \mapsto \mu X. A] \rrbracket(v)$.

4 Implementation

We implement the design of §2 as a prototype extension of Dotty, the production Scala 3 compiler. This section describes how refinement types integrate into the existing compiler architecture (§4.1), how argument hoisting and skolems recover the ANF discipline assumed by the formalization (§4.2), and the lightweight e-graph-based solver that discharges predicate entailment obligations (§4.3). We evaluate compilation overhead on a suite of benchmarks in §4.4.

4.1 Integration

We implement refinement types in Dotty, the production Scala 3 compiler. The resulting change is small. The bulk of the new code lives in a self-contained package and accounts for approximately 2 200 new lines, of which about 600 are for the solver (§4.3). Changes to existing compiler code are limited to about 300 net lines. The main integration points are:

- *Type representation.* Dotty provides a mechanism for defining custom types via type annotations, supporting arbitrary internal representations. We leverage this to attach predicates to types. These predicates are represented using a lightweight expression IR that captures only the subset of Scala expressions valid as predicates (§2.2). Crucially, the leaf nodes of this IR are Scala's

existing path-dependent types, so the compiler's existing substitution and dependent-type machinery is reused automatically.

- *Parsing.* The parser is extended with the two grammar productions presented in §2.1.
- *Subtyping.* A single case is added to the type comparer, corresponding to S-REFINE (Figure 10): when the supertype is $\{x : A \mid p\}$, the comparer checks $<: A$ and delegates predicate entailment to the solver (§4.3).
- *Adaptation.* Dotty's bidirectional type checker has a single point where a synthesized type is reconciled with an expected type. This point gains one new case: when the expected type is a refinement type, it attempts selfification (§2.4), wrapping the expression in $\{x : T \mid x == e\}$.
- *Branch and case facts.* The typer records branch conditions (if) and pattern-match outcomes in the context, and propagates them to the solver. Unlike the formalization, which also stores equality facts for `val` bindings in the typing context (T-LET in Figure 9), the implementation does not need to: the compiler already links each symbol to its definition tree, so the solver can follow references in a predicate and recover definitions on demand, gathering only the facts relevant to the predicate being checked.
- *Argument hoisting.* When a method signature contains refinement types that refer to other parameters, the corresponding arguments are lifted to local `val` bindings (§4.2). This is needed so that run-time checks can refer to the argument without re-evaluating it.
- *Recursive signatures.* When a return type's predicate refers to the method itself, the compiler pre-installs a provisional type without the qualifier to break the cyclic reference.
- *Run-time checks.* The type-test lowering phase compiles pattern matches against $\{x : A \mid p\}$ into a type test against A followed by a run-time evaluation of p [2, 36].

4.2 Hoisting and Skolems

In the formalization (§3), T-APP (Figure 9) requires the argument to be a variable y , so that the return type $B[x \mapsto y]$ is a simple substitution of one variable for another. In practice, Scala allows arbitrary expressions as arguments, so the implementation must recover this structure on demand through two mechanisms.

Argument hoisting. Consider a method `def f(x: Int, y: {v: Int with v > x}): Unit`. When called as `f(impure(), 3.runtimeChecked)`, the run-time check for the second argument must evaluate `impure() > 3`—but `impure()` may have side effects and must not be evaluated twice. The compiler detects that `x` is referenced from a qualifier in another parameter's type and lifts the argument to a local binding. This transformation is triggered whenever a parameter is referenced from a qualifier in another parameter's type or in the return type.

Skolems as on-the-fly ANF. How to apply a dependent method $f: (x: A) \Rightarrow B$ to an arbitrary expression e ? We cannot blindly substitute, because the return type $B[x \mapsto e]$ may not be defined if e is outside of the predicate language fragment. This is a well-known problem in refinement type systems. Borkowski et al. [3] solve it with term-level existential types, giving $f(e)$ the type $\exists x. A \wedge B[x]$ (following Knowles and Flanagan [11]); Hamza et al. [16] use a “let-in-type” construct `let  $x = e$  in  $B$` .

Our formalization takes a simpler route: T-APP requires the argument to be a variable, which is always valid in a predicate. In the implementation, we achieve the same effect using *skolems*: special terms in predicates that refer to specific but unknown values. Each non-variable argument is assigned a unique skolem $?k$. The return type of $f(e)$ is formed by substituting $?k$ for x in B . If e happens to live in the supported fragment, an equality $?k == e$ is added to the equality context.

For example, assume a case class `Range(from: Int, until: Int)` whose constructor returns the selfified type $\{r: \text{Range} \mid r == \text{Range}(\text{from}, \text{until})\}$. When `Range(0, 10)` appears as a receiver (say, in `Range(0, 10).foreach(...)`), it is assigned a fresh skolem $?1$, and `foreach` yields elements of type $\{x: \text{Int} \mid ?1.\text{from} \leq x \ \&\& \ x < ?1.\text{until}\}$. The solver then gets $?1 == \text{Range}(0, 10)$ as an assumption, allowing it to unfold $?1.\text{from}$ to 0 and $?1.\text{until}$ to 10, recovering the bounds needed to verify body expressions like `a(i)`.

In effect, the skolem plays the role of the let-bound variable in the formalization: it names the argument value, and the equality assumption carries the necessary fact. Like let bindings in the formalization, skolems are removed from result types via the `avoid` operation (§3.4) when they would escape their scope.

4.3 Solver

Rather than depending on an external SMT solver—which would add platform-specific binaries and a large unowned dependency to the compiler—we built a self-contained solver (≈ 600 lines) based on e-graphs, lightweight enough to run on every keystroke in an IDE.

The solver decides predicate entailment $P_1 \Rightarrow P_2$, with top-level case splitting for disjunctive assumptions. It collects all relevant facts—qualifier predicates, `val` equalities, and branch conditions—inserts them into an e-graph merged with `true`, and checks whether P_2 becomes equivalent to `true`.

Acyclic e-graph. Our e-graph implementation is similar to the *acyclic e-graphs* used in the Cranelift compiler [9] and described by Zucker [51]: each equivalence class eagerly collapses to a single canonical representative, rather than maintaining the set of all equivalent terms as in standard e-graphs [47]. When two nodes are merged, an *order* function picks the representative (preferring constants, then operations, then constructor applications, then lambdas), and all parent nodes are re-canonicalized. This may trigger further merges—for example, merging $x \equiv 3$ causes $x + y$ to re-canonicalize to $3 + y$, which may match an existing node and cascade into another merge. This repair loop is the core of *congruence closure* [28].

Normalization. During canonicalization, the e-graph applies domain-specific rewrites:

- *Arithmetic.* Integer sums and products are flattened using associativity, like-terms are grouped and constant-folded ($2x + 3x \rightarrow 5x$). Subtraction is rewritten to addition of a negated product ($a - b \rightarrow a + (-1) \cdot b$). Distribution is not applied, to avoid exponential blowup.
- *Boolean logic.* Conjunction and disjunction short-circuit when one operand is a constant. Double negation is eliminated. When a conjunction is merged with `true`, each conjunct is individually merged with `true`; dually for disjunction and `false`. When an equality $a = b$ is merged with `true`, the nodes a and b are themselves merged.
- *Beta reduction.* Applying a lambda to arguments substitutes and re-canonicalizes.
- *Constructor-field reduction.* Selecting a field from a constructor application returns the corresponding argument (`Range(0, 10).from` $\rightarrow$ 0). This is the mechanism that unfolds skolems in the Range example of §4.2.
- *Transitivity of <.* When $a < b$ becomes true, the solver derives $a < c$ for all known c with $b < c$, and symmetrically for lower bounds.

Binding. E-graph nodes may contain lambdas (for predicates passed to higher-order functions). We represent bound variables with de Bruijn indices. When checking $P_1 \Rightarrow P_2$, both predicates are opened by substituting their outermost bound variable with a fresh free variable, akin to the opening operation in locally-nameless representations. This is sound because bound variables are

never merged with any other node—they remain fully abstract, only equal to themselves. Only the opened free variable participates in merges. This approach avoids the complexity of slotted e-graphs [37], which parameterize e-classes by their free variables to support full equality saturation with binders—a generality we do not need.

Trade-offs. This design is strictly more powerful than substitution alone: congruence closure propagates equalities through containing expressions, and cascading repair discovers multi-step simplifications that no single substitution pass would find. It is intentionally less complete than full equality saturation [47]: the *order* function picks one representative per class, so if two equivalent forms of a subexpression would trigger different simplifications, only one is explored. We believe this is an acceptable trade-off for a type checker, where predictable performance matters more than completeness.

4.4 Evaluation

We evaluate our implementation along two axes: compilation time overhead and expressiveness compared to two existing systems for Scala—Schmid and Kunčák’s previous prototype [35] and the Stainless verifier [16]. Stainless is not a refinement type system; it checks pre- and postconditions. It is, however, the de facto tool for verifying Scala programs, even for lightweight properties that refinement types could cover. This makes it a natural point of comparison for our system, on the programs that both systems can verify.

Compilation overhead. We measure single-shot compilation time on a suite of small benchmark programs, comparing each system against its own baseline: the same programs without refinement annotations, compiled with checking disabled. Where possible, we port the same program to Schmid’s system and to Stainless. Baselines are per-system because the systems build on different compiler versions and the ported programs differ slightly in code shape. Benchmarks run under JMH; each benchmark is compiled with 150 single-shot warm-up iterations followed by 20 measurement iterations, and the whole experiment is repeated in 10 independent runs. Each run is reduced to the mean of its measurement iterations; Table 1 reports the mean of the per-run means with 95% confidence intervals (Student’s *t*), and the overhead Δ as the difference between checked and baseline times.

On our benchmarks, our system (labelled “First-class” in the table) adds modest overhead: 0–12% of the baseline compilation time. This is consistently less than the two other systems: on the benchmarks they support, Schmid’s system adds 20–38% and Stainless at least 56%.

To gauge overhead on a realistic codebase, we applied our system to *Fansi* [17], a 1 000-line terminal-color library. The only change was replacing three run-time require checks in a color-index function with a refinement type alias:

```
type ChannelValue = {v: Int with 0 <= v && v < 256}
def trueIndex(r: ChannelValue, g: ChannelValue, b: ChannelValue)
```

All call sites already guard the arguments with the same bounds, so the refinement is verified statically with no other code changes. The compilation time difference is within measurement noise, confirming that the overhead is proportional to the number of refinement checks, not to the size of the program. This experiment also illustrates a key advantage of first-class integration: our system compiles unmodified Scala code without restriction, so refinement types can be introduced incrementally into an existing codebase. By contrast, Stainless is limited to a specific fragment of Scala and fails on code that falls outside its supported subset.

Table 1. Compilation time benchmarks (ms/op, single-shot). Each run (JMH fork) is reduced to its mean; scores are the mean of the per-run means $\pm$ the 95% confidence interval across runs (Student's t). Each platform shows the unchecked baseline time and the absolute overhead of checking; the checked time is base + Δ . Since the baseline and checked samples are independent, the overhead interval is $\sqrt{e_b^2 + e_c^2}$, where e_b and e_c are the baseline and checked intervals.

Benchmark	First-class			Schmid			Stainless		
	LoC	Base (ms)	Δ (ms)	LoC	Base (ms)	Δ (ms)	LoC	Base (ms)	Δ (ms)
arrfold	20	94 $\pm$ 1	3 $\pm$ 2	20	88 $\pm$ 2	30 $\pm$ 2	—	—	—
collect	15	103 $\pm$ 2	4 $\pm$ 3	—	—	—	—	—	—
fansi	530	187 $\pm$ 3	−1 $\pm$ 3	—	—	—	—	—	—
fibMemo	15	104 $\pm$ 1	7 $\pm$ 2	—	—	—	—	—	—
hof1	5	75 $\pm$ 2	3 $\pm$ 2	5	77 $\pm$ 1	18 $\pm$ 1	—	—	—
hof2	8	77 $\pm$ 1	2 $\pm$ 1	8	77 $\pm$ 1	20 $\pm$ 1	—	—	—
intarray	11	94 $\pm$ 1	3 $\pm$ 1	11	75 $\pm$ 1	19 $\pm$ 2	14	68 $\pm$ 1	45 $\pm$ 2
list1	5	95 $\pm$ 1	3 $\pm$ 2	5	83 $\pm$ 2	17 $\pm$ 2	—	—	—
list2	5	95 $\pm$ 1	2 $\pm$ 2	5	83 $\pm$ 2	17 $\pm$ 3	—	—	—
matrix	51	101 $\pm$ 2	11 $\pm$ 2	—	—	—	—	—	—
matrixDims	13	94 $\pm$ 2	4 $\pm$ 2	13	75 $\pm$ 2	31 $\pm$ 2	21	66 $\pm$ 1	93 $\pm$ 1
max	4	70 $\pm$ 1	5 $\pm$ 2	4	74 $\pm$ 1	25 $\pm$ 2	6	49 $\pm$ 1	56 $\pm$ 2
maxGen	11	103 $\pm$ 1	2 $\pm$ 2	—	—	—	—	—	—
maxGenBig	15	109 $\pm$ 2	14 $\pm$ 2	—	—	—	—	—	—
mergeSort	29	95 $\pm$ 1	11 $\pm$ 2	—	—	—	53	62 $\pm$ 2	648 $\pm$ 4
postuple	4	71 $\pm$ 2	6 $\pm$ 2	4	73 $\pm$ 1	18 $\pm$ 2	9	68 $\pm$ 1	36 $\pm$ 1
range	19	99 $\pm$ 1	4 $\pm$ 2	—	—	—	—	—	—
rational	6	71 $\pm$ 1	5 $\pm$ 2	6	75 $\pm$ 1	26 $\pm$ 1	10	67 $\pm$ 1	47 $\pm$ 1
sqrt	9	74 $\pm$ 1	7 $\pm$ 2	9	77 $\pm$ 1	25 $\pm$ 1	14	49 $\pm$ 1	66 $\pm$ 2
sumnat	10	69 $\pm$ 1	6 $\pm$ 1	10	74 $\pm$ 1	21 $\pm$ 2	—	—	—
vec	12	74 $\pm$ 2	6 $\pm$ 2	—	—	—	20	56 $\pm$ 1	136 $\pm$ 2

Impact on code that does not use the feature. Because refinement checking is integrated into the compiler rather than layered on top, a natural concern is whether it affects existing code. To assess this, we run the full Dotty continuous-integration suite with the feature enabled. It passes unchanged: this includes compiling the Scala compiler itself ($\sim 200\,000$ lines of code), its $\sim 10\,000$ compilation tests ($\sim 300\,000$ lines), and a community build of 50 open-source projects ($\sim 700\,000$ lines), indicating that the extension introduces no backward incompatibility. On the official Scala compiler benchmark suite,⁵ enabling the feature slows compilation by at most 4% with the current prototype, on both the 11 “Big” benchmarks, which compile real-world libraries totalling $\sim 70\,000$ lines, and the 20 “Small” benchmarks. In short, when the feature is unused, it changes no behavior and adds only a small compilation-time overhead.

Expressiveness. The three systems differ in which programs they can express. Schmid’s qualifier inference operates in a separate phase from Scala’s type checker, so refinements do not participate in type-argument inference: generic data structures can be instantiated with qualified types only by writing the type arguments explicitly, and inferred instantiations such as the one in Figure 1 are out of reach. Stainless supports richer specifications through require and ensuring clauses with a full SMT backend, but refinements are not types: they do not compose with bounded polymorphism, type inference, or pattern matching. Our system integrates with all of these by construction, since refinement types are ordinary Scala types.

⁵<https://lampepfl.github.io/scala3-benchmarks/>

On the other hand, both Schmid’s system and Stainless have substantially more powerful solvers. Both use SMT-backed solvers that are complete for linear integer arithmetic, algebraic data types, equality, and first-order logic—theories for which our lightweight e-graph solver offers no decision procedure. As a consequence, several of Schmid’s benchmarks that require linear arithmetic reasoning (e.g., `sumnat`, `arrfold`) cannot be verified by our solver, and Stainless can verify functional correctness properties well beyond what refinement types alone can express.

5 Related Work

Mechanized refinement type systems. Hamza et al. [16] mechanize System FR, a dependently-typed calculus with refinement types, polymorphism, recursive types, and dependent pairs, serving as a foundation for the Stainless verifier. Their system is the closest to ours in ambition and was a major inspiration, particularly for the positivity-based treatment of recursive types. Key differences are: (1) System FR enforces *total correctness*, requiring termination proofs for all recursive functions, while our partial-correctness approach avoids this burden; (2) System FR uses dependent pairs as existentials to handle variables going out of scope, while we use avoidance; and (3) System FR uses index-guarded recursive types $\text{Rec}(n)(\alpha \Rightarrow \tau)$, while we use equi-recursive types with fold/unfold subtyping. Borkowski et al. [3] mechanize a refinement type system extending System F in Liquid Haskell. They restrict refinements to base types (no nested refinements), use well-formedness conditions and existential types rather than avoidance, and elegantly axiomatize the implication judgment as eleven abstract properties that an SMT oracle must satisfy for soundness, without committing to a specific solver. We instead leave implication fully semantic (§3.6). Sun et al. [38] mechanize RFJ, a refinement type system for a Featherweight Java-like language, in Coq. They provide both type soundness and logical soundness (characterizing which formulas are valid). Their system lacks parametric polymorphism but handles selfification and class-based subtyping in an object-oriented setting.

Definitional interpreters and semantic soundness. Our mechanization builds on Amin and Rompf’s [1] technique of proving type safety via a fuel-bounded definitional interpreter. One of their key insights is that big-step semantics sidesteps the “bad bounds” problem that plagues small-step proofs for languages with type members. However, their development proves syntactic type safety (progress and preservation) rather than defining semantic types. Owens et al. [31] introduced functional big-step semantics, and Paraskevopoulou et al. [32] use a similar interpreter-based approach with step-indexed logical relations for multi-pass compiler verification. In both systems, fuel is consumed at every evaluation step, whereas ours bounds evaluation depth (§3.2). Semantic typing, interpreting types as predicates or relations on values, has a long history in logical relations [42]. Giarrusso et al. [15] develop a semantic model for DOT, with subtyping and recursive types using step-indexed logical relations and a “later” modality in the type syntax. We avoid step-indexing entirely by requiring strict positivity for recursive type binders (following Hamza et al. [16]), enabling a direct fixpoint construction via intersected approximations (§3.6).

Refinement types in other languages. We reviewed the history of refinement types in the introduction. Flanagan’s hybrid type checking [11] introduced the idea of automatically inserting run-time checks when the static system cannot discharge a refinement obligation. In our system, run-time checks are not inserted automatically: the programmer explicitly requests them via `.runtimeChecked` or pattern matching (§2), which the type system treats as an ordinary conditional branch, so no special rules are needed for the static/dynamic boundary. Moreover, our run-time checks are restricted to first-order predicates, which avoids the blame assignment problem that arises when monitoring higher-order contracts [21]: since functions cannot be checked eagerly, a violation detected at a later call site must be attributed to the party at fault.

Liquid Haskell [44, 45] brings refinement types to Haskell as a separate plugin, with refinement inference running apart from GHC’s own type checking, whereas our refinements are typed and inferred by the compiler itself. Termination also plays a different role: Liquid Haskell by default requires every function to be proven terminating, while we need termination only in the solver, to justify certain normalization and unfolding rules (§2.3). Finally, Liquid Haskell discharges obligations with an SMT solver over decidable theories, while our solver (§4.3) combines e-graphs with normalization rules: it forgoes arithmetic decision procedures, but natively performs beta reduction and limited higher-order reasoning over predicate lambdas. The recent PLEX extension [10] adds normalization—eta-expansion, beta-reduction, and dependent pattern matching—to Liquid Haskell’s Proof by Logical Evaluation layer, precisely to recover higher-order reasoning over a first-order SMT solver.

In the systems programming space, refinement types have been combined with mutation, where the central difficulty is deciding when established predicates must be invalidated: a refinement of a mutable location can be treated as an *invariant* that every write must preserve, or as a flow-sensitive fact that a write updates or invalidates. Under aliasing, knowing which writes affect which facts is itself hard. Refinement type systems for Rust lean on the ownership discipline: Flux [25] permits *strong updates*, changing a location’s refinement, through exclusively owned pointers, but only invariant-preserving *weak updates* through mutable borrows, while Thrust [30] and RefinedRust [13] type a borrow as its current value paired with a *prophecy variable* for its final value. RefinedC [34] and RefinedRust come with foundational soundness proofs in the Iris separation logic, and Verus [24] provides SMT-based verification for Rust using linear ghost types; these systems target memory safety and low-level invariants. Generic refinement types [26] add refinement polymorphism to Flux. The separation checking sketched in §2.2 would give Scala a similar handle on invalidation: like ownership, capabilities [48] with separation checking [49] make writes explicit and un-aliased, identifying the facts a write can invalidate. For TypeScript, Refined TypeScript [46] sidesteps invalidation by allowing refinements to mention object state only through immutable fields, and recovers flow-sensitive reasoning for local variables via an SSA translation.

The Checker Framework [8] brings user-defined *pluggable* type qualifiers to Java: like refinement types, qualifiers attach additional information to types, and like the separate-phase systems above, the checkers run as compiler plugins over Java’s annotation syntax, outside the language’s own type system. Qualifiers are drawn from a fixed vocabulary and checked by dataflow analysis rather than a solver, a trade-off that scales to large codebases and many practical properties, such as verifying array indexing [22], at the cost of expressiveness. Lanzinger et al. [23] blur the line between the two approaches by combining pluggable type qualifiers with deductive verification, discharging obligations that the type checker cannot prove with a program verifier.

6 Conclusion

We have shown that refinement types can be integrated as first-class citizens into the type system of a mainstream programming language. By extending the Scala 3 compiler directly, refinement types interact naturally with subtyping, bounded polymorphism, type inference, and pattern matching, avoiding the two-mental-models problem of separate-phase approaches.

On the theoretical side, we provide the first mechanized soundness proof that combines refinement types with union and intersection types, bounded polymorphism with lower and upper bounds, and positive equi-recursive types. The proof uses a fuel-bounded definitional interpreter with semantic typing in Rocq, and establishes partial-correctness type safety for a core calculus that captures key features of Scala’s type system. On the practical side, the prototype adds refinement types to an industrial-strength compiler with modest changes ($\approx 2\,500$ lines), a self-contained e-graph solver requiring no external dependencies, and compilation overhead that remains low.

Data-Availability Statement

The accompanying artifact [4] contains:

- **Mechanization.** The Rocq formalization of the type system, with its soundness proof.
- **Implementation.** The source of our Dotty extension, also published on Maven and usable from any SBT project with:

```
scalaOrganization := "ch.epfl.lara"
scalaVersion := "3.10.0-RC1-bin-20260720-5f1bae1-NIGHTLY"
scalacOptions += Seq("-language:experimental.qualifiedTypes")
```

- **Benchmarks.** The JMH benchmark suites used for the evaluation, compiling similar programs with our Dotty fork, Schmid’s prototype, and Stainless, together with scripts to run them and collect results.

The same material, with its continued development, is available in the project’s public repository: <https://github.com/mbovel/first-class-refinement-types-in-scala>.

AI-Usage Statement

AI tools were used in the preparation of this paper and its artifacts, primarily Claude Code (Opus 4.5 and 4.6, Fable 5). Theorem and lemma statements were written by the authors, while many proof bodies were generated with AI assistance. The core implementation predates widespread agentic AI tools, but recent bug fixes and refactorings were AI-assisted (AI-assisted commits are explicitly marked in the git history with Co-authored-by: Claude *). GitHub Copilot was also used for code autocompletion, and Google and ChatGPT for general research questions.

Acknowledgments

We thank the LAMP and LARA teams at EPFL for insightful discussions and the prior work that laid the foundation for this paper, and in particular Sébastien Doeraene and Guillaume Martres for their valuable feedback on the Dotty implementation.

References

- [1] Nada Amin and Tiark Rompf. 2017. Type soundness proofs with definitional interpreters. *SIGPLAN Not.* 52, 1 (Jan. 2017), 666–679. doi:10.1145/3093333.3009866
- [2] Quentin Bernet. 2024. *Syntax and Runtime Checks for Qualified Types in Scala 3*. Master’s thesis. École Polytechnique Fédérale de Lausanne (EPFL), Lausanne, Switzerland.
- [3] Michael H. Borkowski, Niki Vazou, and Ranjit Jhala. 2024. Mechanizing Refinement Types. *Proc. ACM Program. Lang.* 8, POPL, Article 70 (Jan. 2024), 30 pages. doi:10.1145/3632912
- [4] Matt Bovel, Viktor Kunčák, and Martin Odersky. 2026. *First-Class Refinement Types in Scala — Artifact*. doi:10.5281/zenodo.21737784
- [5] Luca Cardelli, Simone Martini, John C. Mitchell, and Andre Scedrov. 1991. An extension of system F with subtyping. In *Theoretical Aspects of Computer Software*, Takayasu Ito and Albert R. Meyer (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 750–770.
- [6] Robert Cartwright. 1976. User-Defined Data Types as an Aid to Verifying LISP Programs. In *Third International Colloquium on Automata, Languages and Programming, University of Edinburgh, UK, July 20-23, 1976*, S. Michaelson and Robin Milner (Eds.). Edinburgh University Press, 228–256.
- [7] École Polytechnique Fédérale de Lausanne. 2024. The Scala 3 Language Specification, Version 3.4. §13: Syntax Summary. Retrieved July 21, 2026 from <https://www.scala-lang.org/files/archive/spec/3.4/>, archived at [<https://perma.cc/ENJ6-7FAR>]
- [8] Werner Dietl, Stephanie Dietzel, Michael D. Ernst, Kivanç Muşlu, and Todd W. Schiller. 2011. Building and Using Pluggable Type-Checkers. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE ’11)*. Association for Computing Machinery, New York, NY, USA, 681–690. doi:10.1145/1985793.1985889
- [9] Chris Fallin. 2023. Cranelift’s Acyclic Egraph implementation. Rust crate documentation. Retrieved July 21, 2026 from https://docs.rs/craneflift-egraph/latest/craneflift_egraph/, archived at [<https://perma.cc/T7YL-6QEX>]

- [10] Alessio Ferrarini, Niki Vazou, and Wouter Swierstra. 2026. PLEX: Normalization for Refinement Types. *Proc. ACM Program. Lang.* 10, OOPSLA1 (April 2026), 1321–1347. doi:10.1145/3798248
- [11] Cormac Flanagan. 2006. Hybrid type checking. *SIGPLAN Not.* 41, 1 (Jan. 2006), 245–256. doi:10.1145/1111320.1111059
- [12] Tim Freeman and Frank Pfenning. 1991. Refinement types for ML. *SIGPLAN Not.* 26, 6 (May 1991), 268–277. doi:10.1145/113446.113468
- [13] Lennard Gäher, Michael Sammler, Ralf Jung, Robbert Krebbers, and Derek Dreyer. 2024. RefinedRust: A Type System for High-Assurance Verification of Rust Programs. *Proc. ACM Program. Lang.* 8, PLDI, Article 192 (June 2024), 25 pages. doi:10.1145/3656422
- [14] Catarina Gamboa, Abigail Reese, Alcides Fonseca, and Jonathan Aldrich. 2025. Usability Barriers for Liquid Types. *Proc. ACM Program. Lang.* 9, PLDI, Article 224 (June 2025), 26 pages. doi:10.1145/3729327
- [15] Paolo G. Giarrusso, Léo Stefanescu, Amin Timany, Lars Birkedal, and Robbert Krebbers. 2020. Scala step-by-step: soundness for DOT with step-indexed logical relations in Iris. *Proc. ACM Program. Lang.* 4, ICFP (2020), 114:1–114:29. doi:10.1145/3408996
- [16] Jad Hamza, Nicolas Voirol, and Viktor Kunčák. 2019. System FR: formalized foundations for the stainless verifier. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 166 (Oct. 2019), 30 pages. doi:10.1145/3360592
- [17] Li Haoyi. 2016. Fansi: Scala library for manipulating Fancy Ansi colored strings. Retrieved July 21, 2026 from <https://github.com/com-lihaoyi/fansi>
- [18] Iltotore. 2021. Iron: A compile-time and runtime refinement library for Scala 3. Retrieved July 21, 2026 from <https://github.com/Iltotore/iron>
- [19] Ranjit Jhala. 2019. Why Refinement Types? Retrieved July 21, 2026 from <https://ucsd-progsys.github.io/liquidhaskell/blogposts/2019-10-20-why-types.lhs/>, archived at [<https://perma.cc/7PMH-598D>]
- [20] Ranjit Jhala and Niki Vazou. 2021. Refinement Types: A Tutorial. *Found. Trends Program. Lang.* 6, 3–4 (Oct. 2021), 159–317. doi:10.1561/25000000032
- [21] Matthias Keil and Peter Thiemann. 2015. Blame assignment for higher-order contracts with intersection and union. *SIGPLAN Not.* 50, 9 (Aug. 2015), 375–386. doi:10.1145/2858949.2784737
- [22] Martin Kellogg, Vlastimil Dort, Suzanne Millstein, and Michael D. Ernst. 2018. Lightweight Verification of Array Indexing. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2018)*. Association for Computing Machinery, New York, NY, USA, 3–14. doi:10.1145/3213846.3213849
- [23] Florian Lanzinger, Alexander Weigl, Mattias Ulbrich, and Werner Dietl. 2021. Scalability and Precision by Combining Expressive Type Systems and Deductive Verification. *Proc. ACM Program. Lang.* 5, OOPSLA (Oct. 2021), 1–29. doi:10.1145/3485520
- [24] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 85 (April 2023), 30 pages. doi:10.1145/3586037
- [25] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. 2023. Flux: Liquid Types for Rust. *Proc. ACM Program. Lang.* 7, PLDI, Article 169 (June 2023), 25 pages. doi:10.1145/3591283
- [26] Nico Lehmann, Cole Kurashige, Nikhil Akiti, Niroop Krishnakumar, and Ranjit Jhala. 2025. Generic Refinement Types. *Proc. ACM Program. Lang.* 9, POPL (Jan. 2025), 1446–1474. doi:10.1145/3704885
- [27] K. Rustan M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning*, Edmund M. Clarke and Andrei Voronkov (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 348–370. doi:10.1007/978-3-642-17511-4_20
- [28] Greg Nelson and Derek C. Oppen. 1980. Fast Decision Procedures Based on Congruence Closure. *J. ACM* 27, 2 (April 1980), 356–364. doi:10.1145/322186.322198
- [29] Martin Odersky, Yaoyu Zhao, Yichen Xu, Oliver Bračevac, and Cao Nguyen Pham. 2026. Tracking Capabilities for Safer Agents. arXiv:2603.00991 [cs.AI] <https://arxiv.org/abs/2603.00991>
- [30] Hiromi Ogawa, Taro Sekiyama, and Hiroshi Unno. 2025. Thrust: A Prophecy-Based Refinement Type System for Rust. *Proc. ACM Program. Lang.* 9, PLDI, Article 230 (June 2025), 29 pages. doi:10.1145/3729333
- [31] Scott Owens, Magnus O. Myreen, Ramana Kumar, and Yong Kiam Tan. 2016. Functional Big-Step Semantics. In *Proceedings of the 25th European Symposium on Programming Languages and Systems - Volume 9632*. Springer-Verlag, Berlin, Heidelberg, 589–615. doi:10.1007/978-3-662-49498-1_23
- [32] Zoe Paraskevopoulou, John M. Li, and Andrew W. Appel. 2021. Compositional optimizations for CertiCoq. *Proc. ACM Program. Lang.* 5, ICFP, Article 86 (Aug. 2021), 30 pages. doi:10.1145/3473591
- [33] Patrick M. Rondon, Ming Kawaguchi, and Ranjit Jhala. 2008. Liquid types. *SIGPLAN Not.* 43, 6 (June 2008), 159–169. doi:10.1145/1379022.1375602
- [34] Michael Sammler, Rodolphe Lepigre, Robbert Krebbers, Kayvan Memarian, Derek Dreyer, and Deepak Garg. 2021. RefinedC: automating the foundational verification of C code with refined ownership types. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (Virtual, Canada) (PLDI*

- 2021). Association for Computing Machinery, New York, NY, USA, 158–174. doi:10.1145/3453483.3454036
- [35] Georg Stefan Schmid and Viktor Kuncak. 2016. SMT-based checking of predicate-qualified types for Scala. In *Proceedings of the 2016 7th ACM SIGPLAN Symposium on Scala* (Amsterdam, Netherlands) (SCALA 2016). Association for Computing Machinery, New York, NY, USA, 31–40. doi:10.1145/2998392.2998398
- [36] Valentin Schneeberger. 2024. *Runtime Checks for Qualified Types in Scala 3*. Bachelor’s thesis. École Polytechnique Fédérale de Lausanne (EPFL), Lausanne, Switzerland.
- [37] Rudi Schneider, Marcus Rossel, Amir Shaikhha, Andrés Goens, Thomas Köehler, and Michel Steuwer. 2025. Slotted E-Graphs: First-Class Support for (Bound) Variables in E-Graphs. *Proc. ACM Program. Lang.* 9, PLDI, Article 223 (June 2025), 23 pages. doi:10.1145/3729326
- [38] Ke Sun, Di Wang, Sheng Chen, Meng Wang, and Dan Hao. 2024. Formalizing, Mechanizing, and Verifying Class-Based Refinement Types. In *38th European Conference on Object-Oriented Programming (ECOOP 2024)* (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 313), Jonathan Aldrich and Guido Salvaneschi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 39:1–39:30. doi:10.4230/LIPIcs.ECOOP.2024.39
- [39] Nikhil Swamy, Cătălin Hrițcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, Jean-Karim Zinzindohoue, and Santiago Zanella-Béguelin. 2016. Dependent types and multi-monadic effects in F*. *SIGPLAN Not.* 51, 1 (Jan. 2016), 256–270. doi:10.1145/2914770.2837655
- [40] The Rocq Development Team. 2025. *The Rocq Prover*. <https://rocq-prover.org/>
- [41] Frank S. Thomas. 2015. refined: Refined is a Scala library for refinement types. Retrieved July 21, 2026 from <https://github.com/fthomas/refined>
- [42] Amin Timany, Robbert Krebbers, Derek Dreyer, and Lars Birkedal. 2024. A Logical Approach to Type Soundness. *J. ACM* 71, 6, Article 40 (Nov. 2024), 75 pages. doi:10.1145/3676954
- [43] Niki Vazou, Patrick M. Rondon, and Ranjit Jhala. 2013. Abstract refinement types. In *Proceedings of the 22nd European Conference on Programming Languages and Systems* (Rome, Italy) (ESOP’13). Springer-Verlag, Berlin, Heidelberg, 209–228. doi:10.1007/978-3-642-37036-6_13
- [44] Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. 2014. Refinement types for Haskell. *SIGPLAN Not.* 49, 9 (Aug. 2014), 269–282. doi:10.1145/2692915.2628161
- [45] Niki Vazou, Anish Tondwalkar, Vikraman Choudhury, Ryan G. Scott, Ryan R. Newton, Philip Wadler, and Ranjit Jhala. 2017. Refinement reflection: complete verification with SMT. *Proc. ACM Program. Lang.* 2, POPL, Article 53 (Dec. 2017), 31 pages. doi:10.1145/3158141
- [46] Panagiotis Vekris, Benjamin Cosman, and Ranjit Jhala. 2016. Refinement Types for TypeScript. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Santa Barbara, CA, USA) (PLDI ’16). Association for Computing Machinery, New York, NY, USA, 310–325. doi:10.1145/2908080.2908110
- [47] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and extensible equality saturation. *Proc. ACM Program. Lang.* 5, POPL, Article 23 (Jan. 2021), 29 pages. doi:10.1145/3434304
- [48] Yichen Xu, Oliver Bračevac, Cao Nguyen Pham, and Martin Odersky. 2025. What’s in the Box: Ergonomic and Expressive Capture Tracking over Generic Data Structures. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 334 (Oct. 2025), 28 pages. doi:10.1145/3763112
- [49] Yichen Xu, Oliver Bračevac, Cao Nguyen Pham, Yaoyu Zhao, and Martin Odersky. 2026. System Cappybara: Tracking Capabilities for Separation and Freshness (Extended Version). arXiv:2607.09383 [cs.PL] <https://arxiv.org/abs/2607.09383>
- [50] Jean-Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. 2017. HACl*: A Verified Modern Cryptographic Library. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (Dallas, Texas, USA) (CCS ’17). Association for Computing Machinery, New York, NY, USA, 1789–1806. doi:10.1145/3133956.3134043
- [51] Philip Zucker. 2024. Acyclic Egraphs and Smart Constructors. Retrieved July 21, 2026 from https://www.philipzucker.com/smart_constructor_aegraph/, archived at [<https://perma.cc/3X86-XKRU>]

Received 2026-03-23; accepted 2026-08-06

A More examples

// Example 3: Matrix operations with checked dimensions

```

case class Matrix(width: Pos, height: Pos):
  private val size = width * height
  private val data = CheckedArray(size.runtimeChecked)
  def index(i: Pos with i < height,
           j: Pos with j < width): {res: Pos with res < data.size} =
    (i * width + j).runtimeChecked
  def apply(i: Pos with i < height, j: Pos with j < width): Double =
    data(index(i, j))
  def update(i: Pos with i < height, j: Pos with j < width, x: Double): Unit =
    data(index(i, j)) = x
  def transpose(): {v: Matrix with v.width == height && v.height == width} = ???
  def mul(that: Matrix with that.height == width):
    {v: Matrix with v.width == that.width && v.height == height}
  =
    val res = Matrix(that.width, height)
    forRange(0, height): i =>
      forRange(0, that.width): j =>
        var sum = 0.0
        forRange(0, width): k =>
          sum = sum + this(i, k) * that(k, j)
        res(i, j) = sum
    res
def matricesExample =
  val m1 = Matrix(2, 3)
  val m2 = Matrix(3, 2)
  m1.mul(m2)
  val m1T = m1.transpose()
  m1T.mul(m1)

```

// Example 4: Memoized Fibonacci with a dependent map

```

case class DepMap[K, V](p: (K, V) => Boolean):
  def put(n: K, v: V with p(n, v)): Unit = ???
  def get(n: K): Option[{res: V with p(n, res)}] = ???
def fib(n: Int): {r: Int with r == (if n <= 1 then 1 else fib(n - 1) + fib(n - 2))} =
  if n <= 1 then 1 else fib(n - 1) + fib(n - 2)
val cache: (DepMap[Int, Int] with (cache == DepMap[Int, Int]((k, v) => v == fib(k)))) =
  DepMap[Int, Int]((k, v) => v == fib(k))
def fibMemo(n: Int): {res: Int with res == fib(n)} =
  cache.get(n) match
    case Some(res) => res
    case None =>
      val res: (Int with (res == fib(n))) =
        if n <= 1 then 1 else fibMemo(n - 1) + fibMemo(n - 2)
      cache.put(n, res)
  res

```